

基于守恒型有限体积法与干基质量坐标的 药材热风烘干耦合传热传质建模

摘 要

药材热风烘干需兼顾干燥品质与能耗时长。本文以能量与水分质量守恒为起点，建立一维轴对称柱坐标耦合传热传质模型，并构建统一的守恒型有限体积数值内核，以下四问在同一内核上逐层递进。

针对问题一，在附录 2 常物性、恒定热风下调用上述内核求解预热阶段 ($0 \sim 1800\text{ s}$) 的柱坐标耦合场，用后向 Euler 时间推进得到温度与含水率的时空分布；结果表明温度扰动尚未传到轴心（轴心仅升至约 33.6°C ），失水被限制在表面薄层，轴心含水率仍保持初值 2.55，内部尚未进入干燥，该常物性解构成后续各问的基准参照。

针对问题二，改用附录 3 温度—含水率变物性，令 ρ, c_p, k, D 随场变化并在同一 Picard 回路内联立更新，构成双向强耦合模型求解 $0 \sim 3\text{ h}$ 全过程；发现温度场全场温差已不足 0.12°C 、近准稳态，而轴心含水率仅降到 1.77，据此揭示传质比传热慢一到两个数量级（时间尺度比 τ_m/τ_{th} 由 30 增至 211），为终点判据只建在含水率场上提供依据。

针对问题三，引入附件 1 的时变热风边界，以轴心含水率的首次穿越判据 $\max_r C < C_{th}$ 求解达标时长，并采用 Kirchhoff 积分平均处理界面扩散、避免调和平均被最小值锁死；算得达标时长 $t^* = 57.262\text{ h}$ ，传质毕奥数由约 1.4 增至远大于 1，干燥由外部对流控制转为内部扩散控制，解释了干燥曲线后段的拖尾。

针对问题四，进一步计入附件 2 的体积收缩，借助干基质量不变量 $\rho_s R^2 = \sigma$ 经 Landau 变换把移动边界几何效应凝聚为单一时变因子 $1/R(t)^2$ 并沿用同一内核求解；算得达标时长 $t^* = 50.898\text{ h}$ ，较固定域基准缩短约 11%，纯径向收缩假设经附件 2 独立检验半径预测偏差仅 1.10%。全模型经 Bessel 解析解、Richardson 收敛阶与机器精度守恒审计验证，灵敏度分析显示结果对环境外推稳健 ($< 0.6\%$)，而忽略收缩会使 t^* 虚高 1.5 倍。

关键字： 耦合传热传质 有限体积法 Kirchhoff 变换 移动边界 干基质量坐标

一、问题重述与分析

1.1 问题背景与任务

药材热风烘干是把切成圆柱形的鲜药材置于恒定或时变的热风环境中，通过表面对流同时带走热量与水分，使内部含水率逐步降到贮藏安全线的过程。药材可近似为半径 $R_0 = 2\text{ cm}$ 、长 $L = 25\text{ cm}$ 的实心圆柱；因 $L \gg R_0$ ，端面效应可忽略，热质输运沿径向占主导，问题退化为一维轴对称柱坐标下的耦合传热传质过程。初始温度 $T_0 = 28^\circ\text{C}$ 、初始干基含水率 $C_0 = 2.55$ ，安全含水率阈值 $C_{\text{th}} = 0.15$ 。题目给出三套物性附录（附录 2 常物性、附录 3 变物性、附录 4 变物性且伴随体积收缩）与两份实测附件（附件 1 热风温度与空气含湿量时序、附件 2 半径随时间的收缩数据），要求依次完成：

1. **问题一**，在附录 2 常物性、恒定热风条件下，给出预热阶段（ $0 \sim 1800\text{ s}$ ）药材内部温度场与含水率场的时空分布；
2. **问题二**，改用附录 3 温度—含水率双向耦合的变物性，刻画 $0 \sim 3\text{ h}$ 内温度场与含水率场的全过程演化；
3. **问题三**，结合附件 1 的时变热风，求内部各点含水率首次全部降至 C_{th} 以下所需的干燥时长；
4. **问题四**，进一步计入附件 2 观测到的体积收缩，建立随时间缩小的移动边界模型，重新给出达标时长，并与问题三比较收缩的影响。

1.2 整体建模思路

四问共享同一条守恒律骨架：由能量守恒与水分质量守恒，在柱坐标下得到一对以温度 T 与干基含水率 C 为状态量的非稳态抛物型方程，表面施加 Robin 型对流换热与对流传质边界，中心施加轴对称零通量条件^[8, 20]。四问的差别只体现在**物性是否随场变化、边界驱动是否时变以及求解域是否随时间收缩**三个层次的递进，因而适合“先建公共内核、再逐问增量扩展”的组织方式：把守恒型离散格式、Kirchhoff 界面平均与时间推进统一放入公共章（第四、节），各问题章只描述本问新增或改变的部分。由于表面既是升温入口又是失水出口，轴心因对称而通量为零、响应最慢，故“内部各点均达标”的判据必然由轴心含水率控制，这一观察贯穿后两问的终点时刻计算。以下按题号分别说明各问的难点与建模路线。

1.3 问题一的分析

问题一在附录 2 常物性、恒定热风下刻画预热阶段的耦合场。难点有二：一是柱坐标轴心处散度算子含 $1/r$ 奇点，需要一种在轴心天然退化、不必人工正则化的离散方式；二是传热与传质虽同为抛物型，却在此阶段呈现截然不同的推进深度。本问作为全

篇基准，重点在于给出温度场与含水率场的时空分布，并据此判断内部是否已进入实质干燥。

1.4 问题二的分析

问题二改用附录 3 的温度—含水率变物性， ρ, c_p, k, D 均随场演化，形成双向强耦合。难点在于非线性物性与两个守恒方程的联立求解：温度经物性反过来影响扩散，含水率又改变热容与导热。需在同一迭代回路内联立更新 T 与 C ，并辨识传热与传质的时间尺度差异，为后续把终点判据只建立在含水率场上提供依据。

1.5 问题三的分析

问题三引入附件 1 的时变热风边界，求内部各点含水率首次全部降至安全线以下的时长。难点在于时变边界驱动下的首次穿越型终点判据，以及扩散系数随含水率跨越约一个数量级所导致的界面平均失真——简单的算术或调和平均会显著偏离真实通量。需采用守恒型的界面处理，才能避免干燥曲线后段出现非物理的拖尾。

1.6 问题四的分析

问题四进一步计入附件 2 观测到的体积收缩，求解域随时间缩小，成为移动边界问题。难点在于收缩带来的动网格困难与几何度规的时变。宜借助于物质总量守恒，把移动边界的几何效应凝聚到固定计算域上的一个时变因子中，从而避免直接在收缩物理域上离散；同时须用观测数据独立检验“纯径向收缩”这一关键假设，以免循环论证。

二、模型假设

结合题面已给条件，仅对题面未完全规定、而建模必须补足的现实简化作如下假设：

- (1) **一维轴对称**。因 $L = 25\text{ cm} \gg R_0 = 2\text{ cm}$ ，端面散失相对侧面可忽略，温度与含水率只沿半径 r 变化， $\partial_\theta(\cdot) = \partial_z(\cdot) = 0$ 。这使三维问题降为一维，保留了径向输运这一主导机制。
- (2) **连续介质与局部平衡**。药材视为均质各向同性多孔介质，同一微元内固相、液态水与热量处于局部热力学平衡，可用单一温度 T 与干基含水率 C 描述其状态。
- (3) **Fick 型扩散与 Fourier 型导热**。内部水分迁移服从以 C 为势的 Fick 第二定律，热量传递服从 Fourier 定律；扩散系数 D 与导热系数 k 按各附录设定随 C, T 变化。
- (4) **对流型表面边界**。表面失水与散热速率正比于表面与热风之间的含水率差、温差，即 Robin 边界，传质、传热系数 h_m, h 取为常数；这刻画了外部对流阻力的作用^[2]。
- (5) **潜热不单列**。题面未给蒸发潜热 L ，基准模型不显式扣除表面蒸发吸热，将其影响留待灵敏度分析定量评估（见第十、节），以避免引入未给定参数。

(6) 纯径向均匀收缩（仅问题四）。附件 2 的体积收缩按半径各向同性缩小处理，干物质总量守恒，收缩速率由附件 2 数据经单调插值确定，不额外假设收缩动力学。

三、符号说明

下表收录跨章节反复使用的核心符号，一次性下标与临时派生量在首次出现处就地说明。

符号	含义	单位
r, t	径向坐标、时间	m, s
$R_0, R(t)$	初始半径、 t 时刻半径（问题四移动边界）	m
T, T_{air}	药材温度、热风温度	°C
C, C_{air}	干基含水率、空气等效含水率	1
C_0, C_{th}	初始含水率、安全含水率阈值	1
C_s, T_s	表面含水率、表面温度	1, °C
ρ, c_p	湿基密度、比热容	$\text{kg m}^{-3}, \text{J kg}^{-1} \text{K}^{-1}$
k	导热系数	$\text{W m}^{-1} \text{K}^{-1}$
D	水分扩散系数	$\text{m}^2 \text{s}^{-1}$
h, h_m	对流换热系数、对流传质系数	$\text{W m}^{-2} \text{K}^{-1}, \text{m s}^{-1}$
T_K	绝对温度 $T + 273.15$	K
α	热扩散率 $k/(\rho c_p)$	$\text{m}^2 \text{s}^{-1}$
Bi_m	传质毕奥数 $h_m R_0 / D$	1
$\Phi(C)$	扩散系数的 Kirchhoff 积分势	$\text{m}^2 \text{s}^{-1}$
ξ	固定域无量纲坐标 $r/R(t)$	1
σ	干基质量不变量 $\rho_s R^2$	kg m^{-1}
w_j	控制体柱坐标测度权	1
t^*	内部各点首次全部达标的时刻	h

四、公共建模内核：守恒型柱坐标耦合场的有限体积求解

四个子问题共享同一套守恒律、离散格式与时间推进策略。本节一次性建立这一公共内核，后续问题章只在此基础上说明”保留了什么、新增了什么、结果如何改变”，不再重复整套推导。四问在同一内核上的递进关系与总体技术路线见图 1。

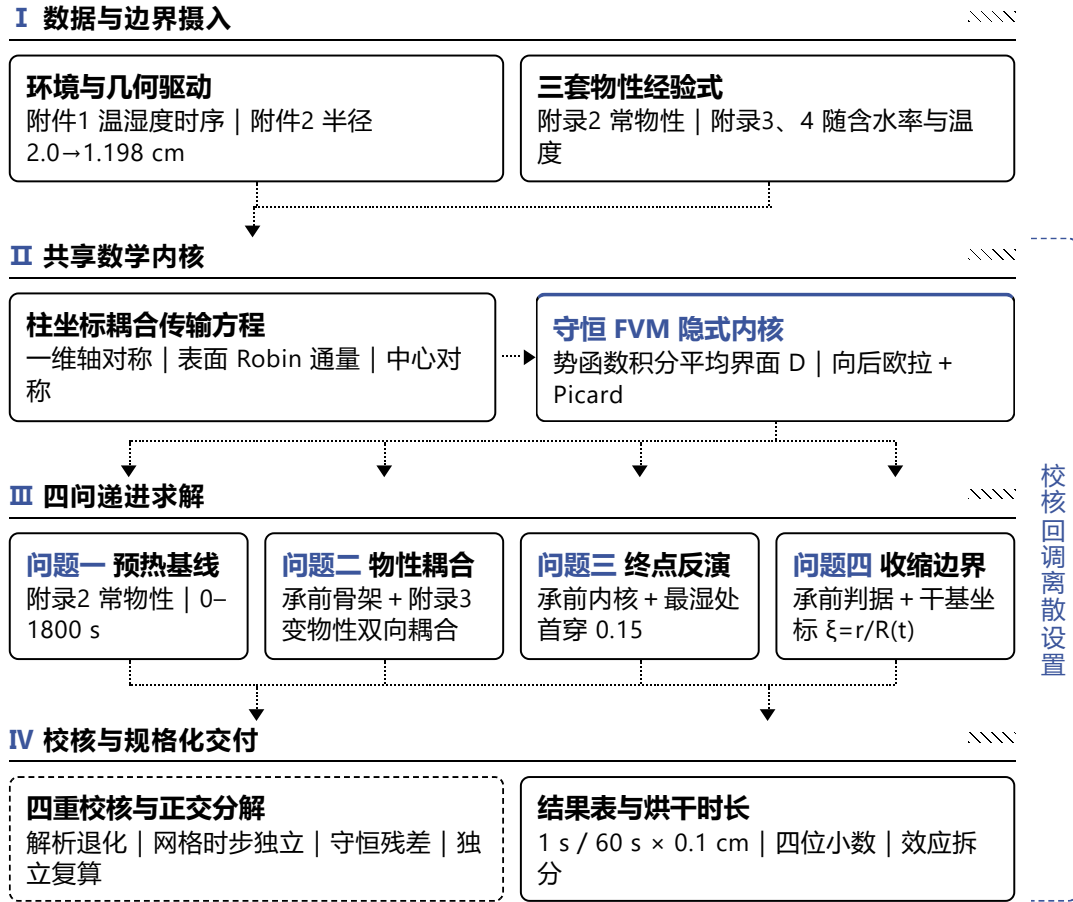


图 1 总体技术路线：从守恒律到公共数值内核，再逐问增量扩展

图 1 表明四问并非彼此独立的四个模型，而是同一守恒型有限体积内核在”物性—边界—求解域”三个层次上的递进，故各问题章只描述本问新增或改变的部分。

4.1 控制方程与定解条件

以柱坐标轴心为原点，取半径 $r \in [0, R_0]$ 。对任意以轴为中心、半径 r 的同轴圆柱控制面，能量守恒与水分质量守恒给出通量的散度形式；在轴对称、忽略轴向与周向输运后，得到一对非稳态抛物型方程：

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \quad \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right), \quad 0 < r < R_0. \quad (1)$$

式 (1) 中左端为单位体积的热/水分储量变化率，右端 $\frac{1}{r} \partial_r(r \cdot)$ 是柱坐标下的散度算子， k, D 分别为导热系数与水分扩散系数。表面 $r = R_0$ 处，内部扩散/导热通量必须与外部对流带走的水分/热量平衡，得 Robin 边界；轴心 $r = 0$ 处由对称性得零通量：

$$-D \frac{\partial C}{\partial r} \Big|_{R_0} = h_m (C_s - C_{\text{air}}), \quad -k \frac{\partial T}{\partial r} \Big|_{R_0} = h (T_s - T_{\text{air}}), \quad \frac{\partial C}{\partial r} \Big|_0 = \frac{\partial T}{\partial r} \Big|_0 = 0. \quad (2)$$

初始条件为 $T(r, 0) = T_0 = 28^\circ\text{C}$ 、 $C(r, 0) = C_0 = 2.55$ 。几何与边界条件的对应关系见图 2。

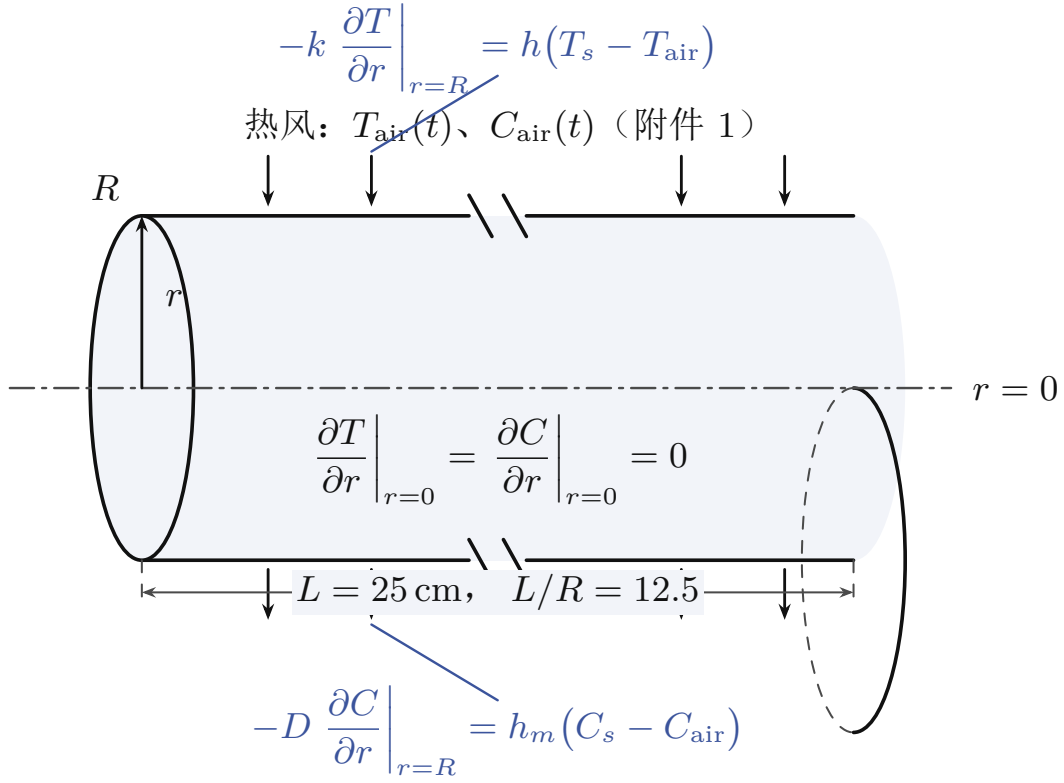


图 2 圆柱几何与柱坐标边界条件：表面 Robin、轴心零通量、端面忽略

图 2 明确了三类边界的物理归属：表面同时承担对流换热与对流传质，轴心因对称退化为绝热绝湿面，端面则因长径比大而被忽略。式 (1)–(2) 是四问共用的连续模型，各问的差异仅在于 ρ, c_p, k, D 与 $T_{\text{air}}, C_{\text{air}}$ 的具体形式。热量与水分的输运路径见图 3。

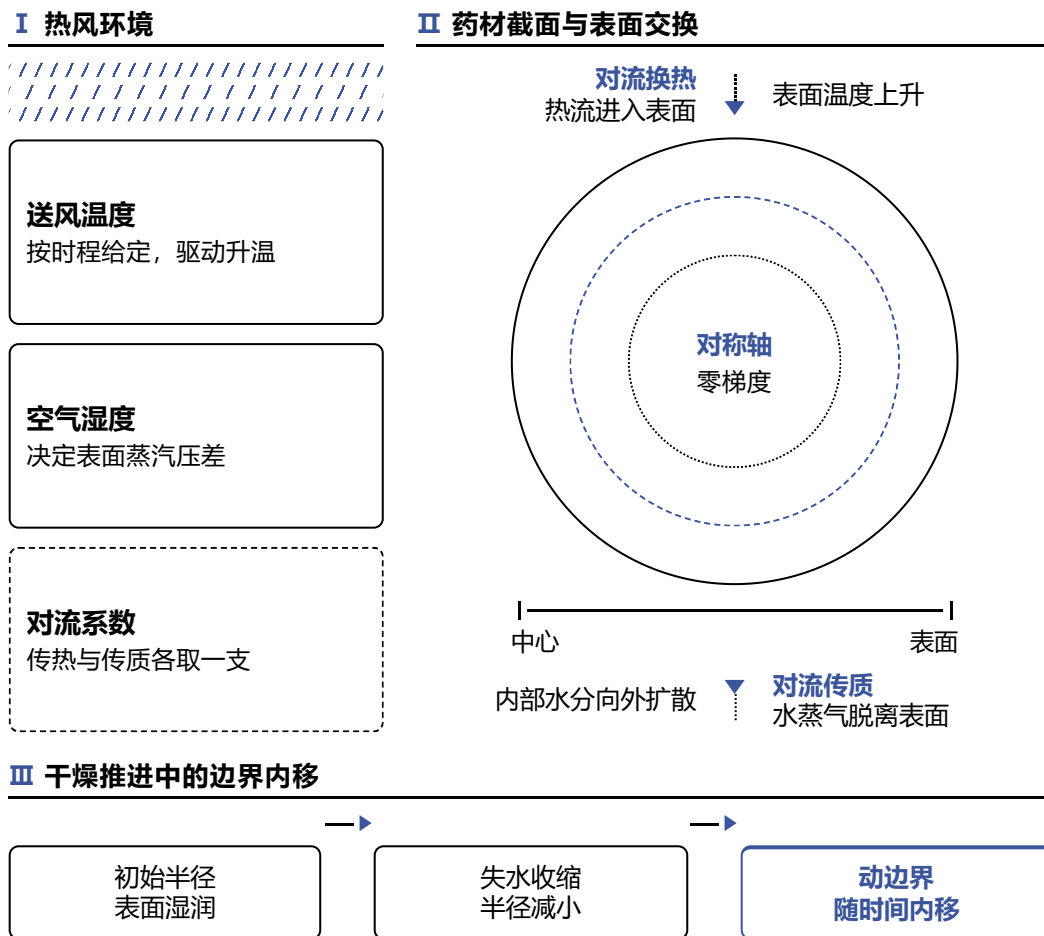


图3 热风烘干中的表面对流换热、对流传质与内部径向扩散

图3表明：热风以对流换热系数 h 向表面供热，水分以对流传质系数 h_m 从表面逸出，内部则由导热与扩散把边界扰动向轴心传播；轴心响应最慢，是“内部各点均达标”判据的控制点。

4.2 三套物性与扩散系数的量级差异

三套附录给出的物性形式不同：附录2为常物性；附录3、附录4均取 $\rho = \rho_0 + \rho_1 C$ 、 $c_p = c_{p0} + c_{p1} \frac{C}{C+1}$ 、 $k = k_0 + k_1 \frac{C}{C+1}$ ，扩散系数则统一采用 Arrhenius 型

$$D(C, T) = D_0 \exp\left(-\frac{a}{C}\right) \exp\left(-\frac{E}{T_K}\right), \quad T_K = T + 273.15, \quad (3)$$

其中 $\exp(-a/C)$ 刻画低含水率时自由水减少导致的扩散受阻， $\exp(-E/T_K)$ 为温度激活项， $E = 3850 \text{ K}^{[16, 17]}$ 。附录3取 $D_0 = 2.4 \times 10^{-3}$ ， $a = 0.45$ ，附录4取 $D_0 = 4.2 \times 10^{-4}$ ， $a = 0.30$ ，各附录完整物性见表2。三套扩散系数的量级对比与附录3的二维响应面见图4与图5。

表 2 三套附录物性模型对照

物性量	附录 2（问题 1）	附录 3（问题 2、3）	附录 4（问题 4）
密度 $\rho / (\text{kg} \cdot \text{m}^{-3})$	820	$650 + 128 C$	$760 + 90 C$
比热 $c_p / (\text{J} \cdot \text{kg}^{-1} \text{K}^{-1})$	2600	$1450 + 2736 \frac{C}{C+1}$	$1850 + 2150 \frac{C}{C+1}$
导热 $k / (\text{W} \cdot \text{m}^{-1} \text{K}^{-1})$	0.36	$0.21 + 0.38 \frac{C}{C+1}$	$0.12 + 0.20 \frac{C}{C+1}$
扩散系数 $D / (\text{m}^2 \cdot \text{s}^{-1})$	$7 \times 10^{-9} e^{-0.89/C}$	$2.4 \times 10^{-3} e^{-0.45/C} e^{-3850/T_K}$	$4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T_K}$
D 初始值 ($C_0, 28^\circ\text{C}$)	4.938×10^{-9}	5.642×10^{-9}	随工况
D 终点值 ($C_{\text{th}}, 50^\circ\text{C}$)	—	8.001×10^{-10}	随工况
温度依赖性	无	Arrhenius	Arrhenius
边界域	固定 R_0	固定 R_0	动边界 $R(t)$

注： C 为干基含水率 ($\text{kg} \cdot \text{kg}^{-1}$)， T_K 为开尔文温度。共用参数： $R_0 = 2 \text{ cm}$ ， $L = 25 \text{ cm}$ ， $T_0 = 28^\circ\text{C}$ ， $C_0 = 2.55$ ，达标阈值 $C_{\text{th}} = 0.15$ ，对流换热 $h = 25 \text{ W} \cdot \text{m}^{-2} \text{K}^{-1}$ ，对流传质 $h_m = 8 \times 10^{-7} \text{ m} \cdot \text{s}^{-1}$ 。 D 锚点值由上式复算，与题面校核点一致。

表 2 给出的锚点表明：初始状态 $D(C_0, 28^\circ\text{C})$ 在附录 2、附录 3 分别为 4.938×10^{-9} 与 $5.642 \times 10^{-9} \text{ m}^2/\text{s}$ ，量级相当；但降到阈值附近 $D(C_{\text{th}}, 50^\circ\text{C})$ 骤降至 8.001×10^{-10} ，说明干燥末期扩散显著变慢，这正是干燥曲线后段拖尾的根源。

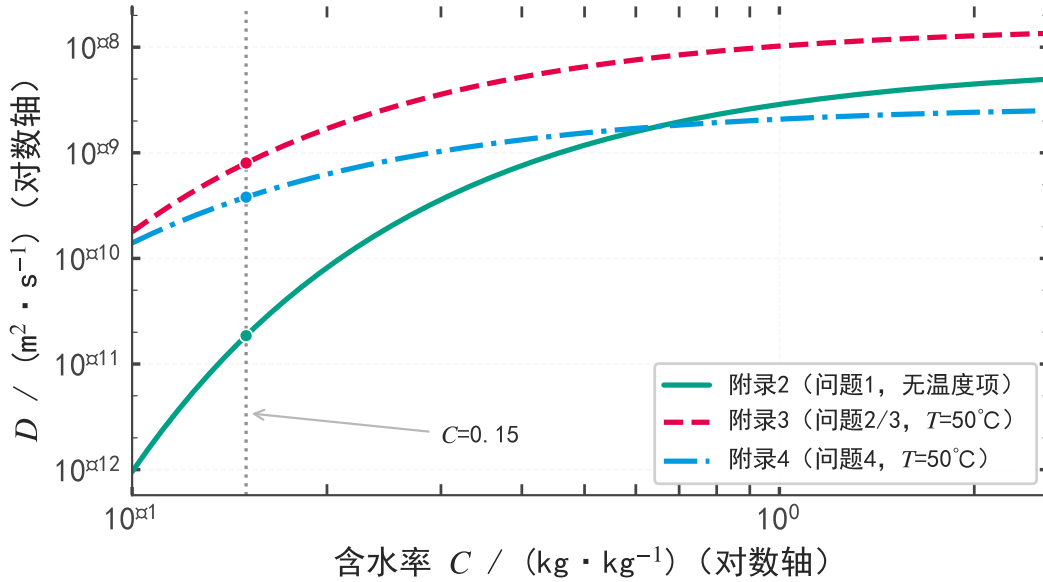


图 4 三套附录扩散系数在 $C \in [C_{\text{th}}, C_0]$ 上的量级对比

图 4 显示，随含水率从 C_0 降到 C_{th} ，扩散系数跨越约一个数量级，且三套物性的下降斜率因 a 不同而分化： a 越大（附录 2 的等效 $a = 0.89$ ），末期抑制越强。这一强非线性使得直接对 $D \partial_r C$ 作简单平均会显著失真，必须在离散时对界面扩散通量作专门处理，详见 4.4 节。

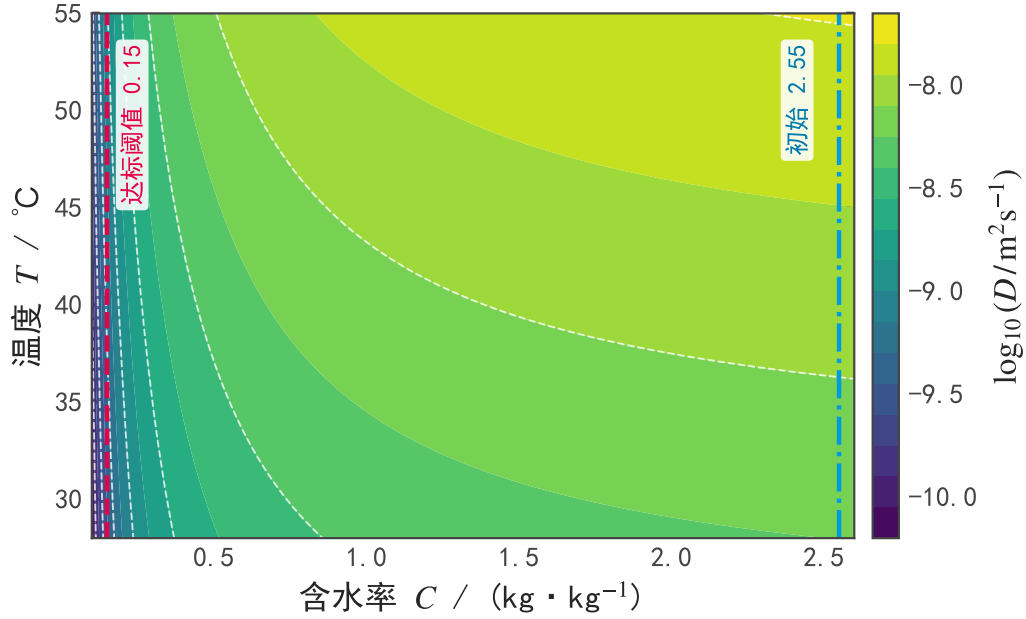


图5 附录3 扩散系数 $D(C, T)$ 在含水率—温度平面上的二维响应面

图5进一步表明 D 对 C 的敏感度远高于对 T ：等值线沿含水率方向密集、沿温度方向平缓，说明在本题温区（28 ~ 50 °C）内，含水率是控制扩散的主因，温度主要通过 Arrhenius 项作二阶调节。这为问题三、四把终点判据建立在含水率场上（而非温度场）提供了依据。

4.3 守恒型有限体积离散

把 $[0, R_0]$ 划分为 N 个控制体，节点 r_j 位于控制体形心，控制面位于 $r_{j\pm 1/2}$ 。对式 (1) 在第 j 个环形控制体上作体积分，利用散度定理把体积分化为控制面上的通量差，得到严格守恒的半离散格式（以 C 为例， T 同构）：

$$w_j \frac{dC_j}{dt} = \frac{2}{R_0^2} \left[r_{j+\frac{1}{2}} \hat{D}_{j+\frac{1}{2}} \frac{C_{j+1} - C_j}{\Delta r} - r_{j-\frac{1}{2}} \hat{D}_{j-\frac{1}{2}} \frac{C_j - C_{j-1}}{\Delta r} \right], \quad w_j = \xi_{j+\frac{1}{2}}^2 - \xi_{j-\frac{1}{2}}^2, \quad (4)$$

其中 $\xi = r/R_0$ ，柱坐标测度权 w_j 满足 $\sum_j w_j = 1$ （等价于 $\sum_j w_j - \pi R_0^2$ 归零，见表5）。式 (4) 的关键在于：最内控制体的内侧控制面面积 $r_{1/2} \rightarrow 0$ 结构性地为零，因而轴心奇点 $1/r$ 被离散格式天然消去，无需对 $\frac{1}{r} \partial_r$ 作洛必达展开或人工正则化。网格与控制体通量平衡见图6。

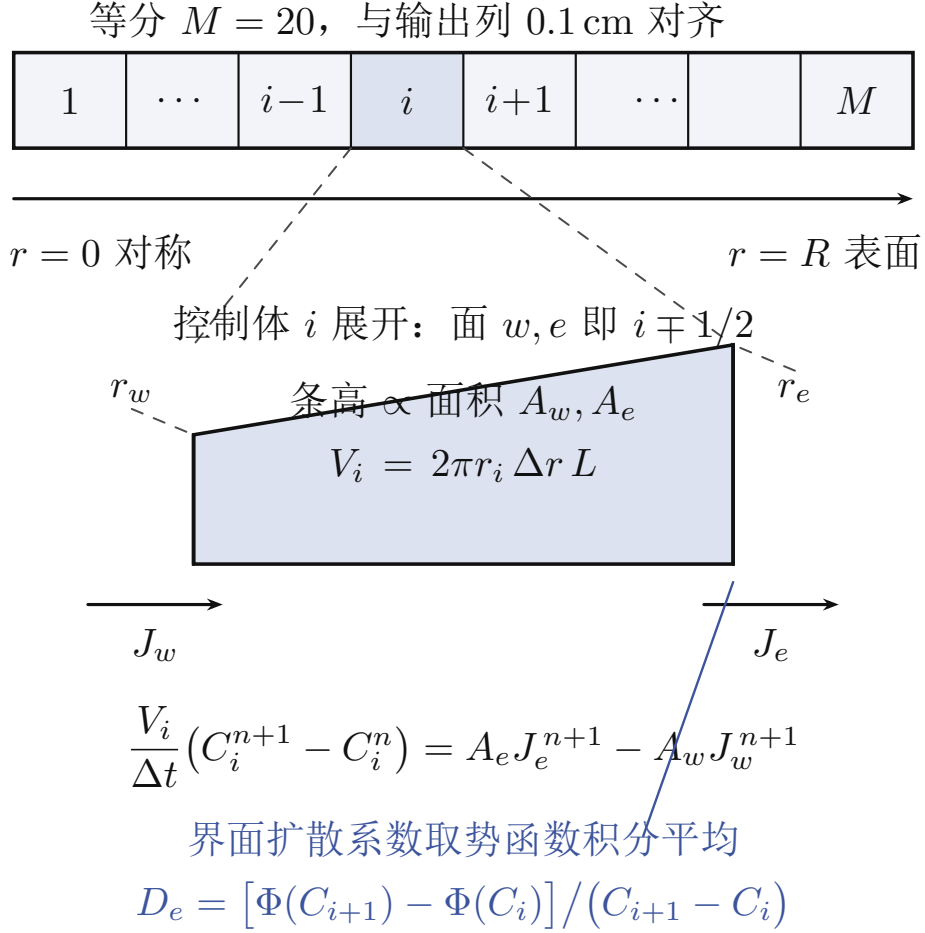


图 6 有限体积网格：节点、控制面与环形控制体上的进出通量平衡

图 6 显示每个控制体的净储量变化等于其两侧控制面的通量差，逐控制体求和后内部通量成对抵消，只剩表面对流项，从而在离散层面精确复现连续守恒律。这一守恒性在第九、节被量化为 $10^{-13} \sim 10^{-16}$ 量级的机器精度残差。

4.4 界面扩散系数的 Kirchhoff 积分平均

式 (4) 需要控制面处的等效扩散系数 $\hat{D}_{j\pm 1/2}$ 。由于 $D(C)$ 随 C 跨越约一个数量级（图 4），常用的算术平均会高估、调和平均会被最小值锁死。为在界面处保持通量 $D \partial_r C = \partial_r \Phi(C)$ 的一致性，引入 Kirchhoff 势 $\Phi(C) = \int^C D(s) ds^{[5, 12]}$ 。对 Arrhenius 型式 (3) 的含水率因子解析积分得

$$\Phi(C) = C e^{-a/C} - a E_1\left(\frac{a}{C}\right), \quad \hat{D}_{j+\frac{1}{2}} = \frac{\Phi(C_{j+1}) - \Phi(C_j)}{C_{j+1} - C_j}, \quad (5)$$

其中 $E_1(\cdot)$ 为指数积分函数，数值上按其标准级数与渐近展开求值。式 (5) 的界面平均保证了跨界面扩散通量守恒，其对达标时刻的影响见图 7 与第十、节：改用算术平均仅使 t^* 变化 -1.80% ，而调和平均因被最小值锁死产生 557.76 h 的非物理结果。

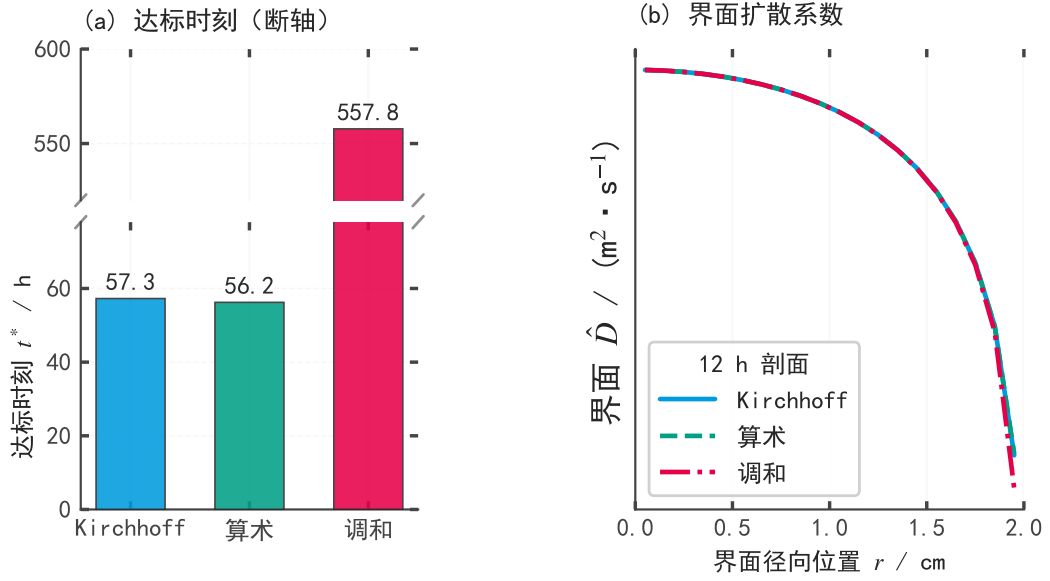


图 7 界面扩散系数平均方式 (Kirchhoff、算术、调和) 对达标时刻的影响

图 7 直观显示调和平均在 D 跨量级处严重偏离，而 Kirchhoff 积分平均与算术平均接近且更符合通量守恒，故全文界面扩散系数统一采用式 (5)。

4.5 时间推进：后向 Euler 与 Picard 内迭代

物性 ρ, c_p, k, D 依赖待求的 T, C ，式 (4) 是强非线性刚性方程组。为兼顾稳定性与守恒性，采用 L -稳定的后向 Euler 格式 ($\theta = 1$)，并在每个时间步用 Picard 迭代把非线性物性冻结更新^[7]：

$$\frac{\mathbf{U}^{n+1,(m+1)} - \mathbf{U}^n}{\Delta t} = \mathcal{L}(\mathbf{U}^{n+1,(m)}) \mathbf{U}^{n+1,(m+1)}, \quad \|\mathbf{U}^{(m+1)} - \mathbf{U}^{(m)}\|_{\infty} < \varepsilon, \quad (6)$$

其中 $\mathbf{U} = (T, C)$ 在同一迭代回路内联立更新以保持双向耦合， $\mathcal{L}$ 为按上一迭代场组装的三对角算子，收敛容差 $\varepsilon = 10^{-11}$ 、最大迭代 40 次。实测每步 Picard 残差降至 9.998×10^{-12} (表 5)。此外，Arrhenius 项在 Kelvin 下计算并施加 $200 < T_K < 500$ 的物理性断言，避免温度越界导致的数值发散。

至此，公共内核（守恒律—有限体积—Kirchhoff 界面—后向 Euler/Picard）建立完毕。以下四章分别说明各问在此内核上的具体设定与结果。

五、问题一：常物性柱坐标耦合场的预热阶段温度与水分分布

5.1 本问设定与建模

问题一采用附录 2 常物性、恒定热风 ($T_{\text{air}} = 50^\circ\text{C}$)，求预热阶段 $0 \sim 1800\text{s}$ 内的温度场与含水率场。此时式 (1) 的系数 ρ, c_p, k, D 均为常数，方程退化为经典的柱坐标

线性热扩散/物质扩散对，热扩散率 $\alpha = k/(\rho c_p) = 1.6886 \times 10^{-7} \text{ m}^2/\text{s}$ ，对应特征热时间 $\tau_{\text{th}} = R_0^2/\alpha \approx 2368.9 \text{ s}$ 。求解直接调用公共内核（式 (4)–(6)），因常物性故 Picard 迭代一步即收敛。

5.2 结果与分析

图 8 给出温度场与含水率场在 (r, t) 平面上的时空演化。

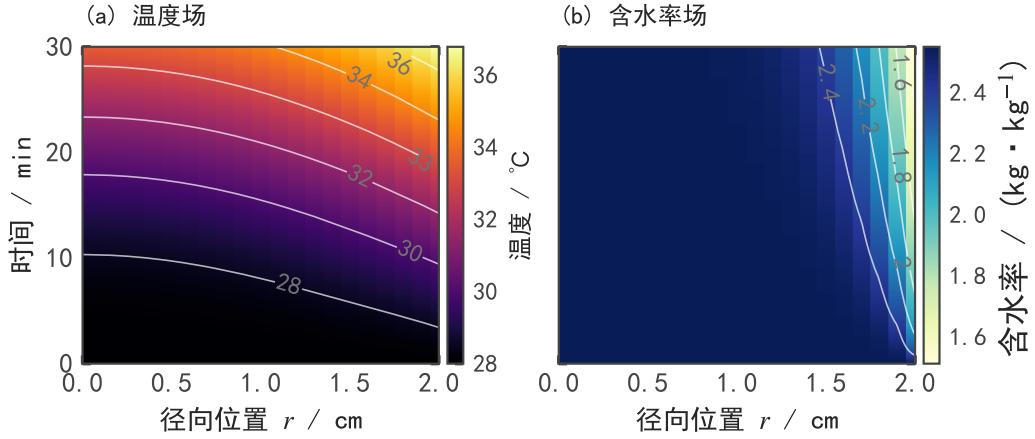


图 8 问题 1 温度场（左）与含水率场（右）在预热阶段的时空演化

由图 8 可见，温度扰动自表面向轴心推进，1800 s 时轴心温度仅由 28°C 升到约 33.6°C ，尚未达到热风温度，印证预热阶段的特征时间 $\tau_{\text{th}} \approx 2369 \text{ s}$ 与观测时窗相当、升温未完成。含水率场则几乎只在近表面薄层内下降，轴心含水率仍保持在 $C_0 = 2.55$ 。径向剖面见图 9。

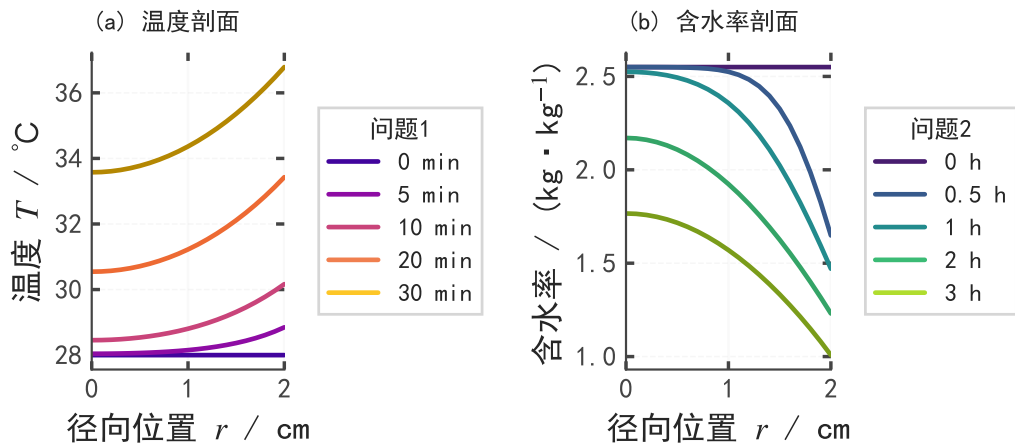


图 9 问题 1 不同时刻温度剖面（左）与问题 2 含水率剖面（右）的径向分布

图 9 左侧的温度剖面显示 1800 s 时沿半径由轴心到表面为 33.58, 33.77, 34.37, 35.36, 36.79 °C, 呈单调升高的边界层型分布; 对应含水率由轴心到表面为 2.5500, 2.5496, 2.5376, 2.3763, 1.5125, 即只有最外一到两层含水率明显下降, 轴心几乎未失水。这说明预热阶段传质被限制在表面薄层, 内部尚未“感知”到干燥, 也解释了为何该阶段不宜用于反演内部含水率——内部信息尚未从边界传入。问题一的结果构成后续变物性、时变边界与移动边界模型的基准解, 各问最终数值汇总见表 4。

六、问题二：变物性双向强耦合下的全过程温度与水分场

6.1 相对问题一的增量：变物性与双向耦合

问题二保留问题一的柱坐标控制方程 (1) 与恒定热风, 但把附录 2 常物性替换为附录 3 变物性: $\rho = 650 + 128C$ 、 $c_p = 1450 + 2736 \frac{C}{C+1}$ 、 $k = 0.21 + 0.38 \frac{C}{C+1}$, 扩散系数按式 (3) 取 $D_0 = 2.4 \times 10^{-3}$, $a = 0.45$ 。此时含水率 C 通过 ρ, c_p, k 影响温度场, 温度 T 又通过 Arrhenius 项 $\exp(-E/T_K)$ 影响扩散、进而改变含水率, 构成双向强耦合。求解时 T, C 在同一 Picard 回路内联立更新 (式 (6)), 观测时窗取 0 ~ 3 h。

6.2 结果与分析

图 10 给出含水率场的时空演化与边缘剖面。

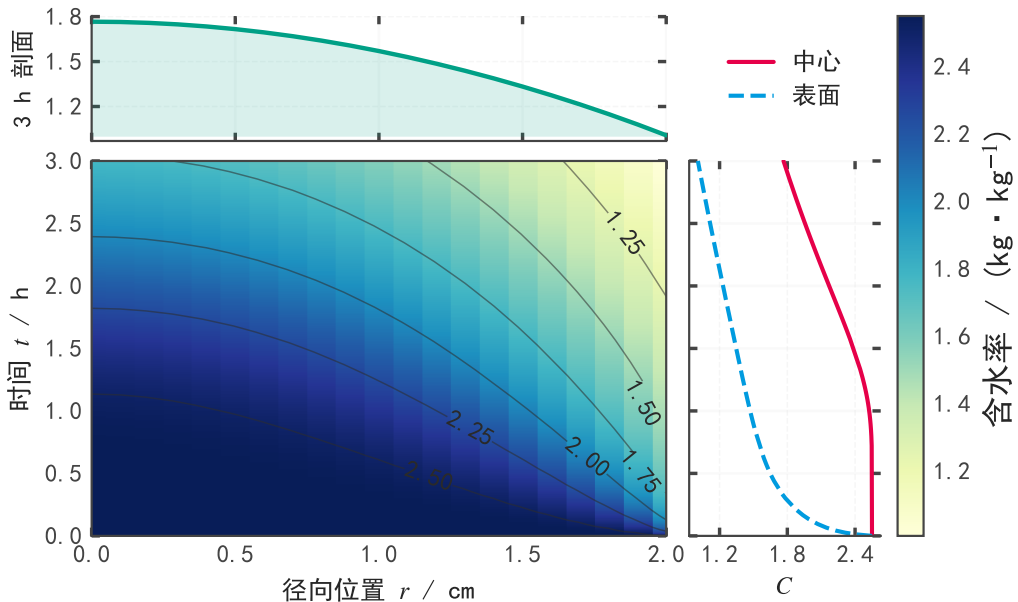


图 10 问题 2 含水率场时空演化与不同时刻的边缘剖面

由图 10, 3 h 时轴心含水率降至 1.7662、表面降至 1.0078, 沿半径由轴心到表面为

1.7662, 1.7165, 1.5701, 1.3331, 1.0078（对应问题 2 剖面见图 9 右）；与问题一相比，失水已从表面薄层推进到全径，但距阈值 0.15 仍有量级差距，说明 3h 远未达标，为问题三的长时程计算埋下伏笔。温度场等值线见图 11。

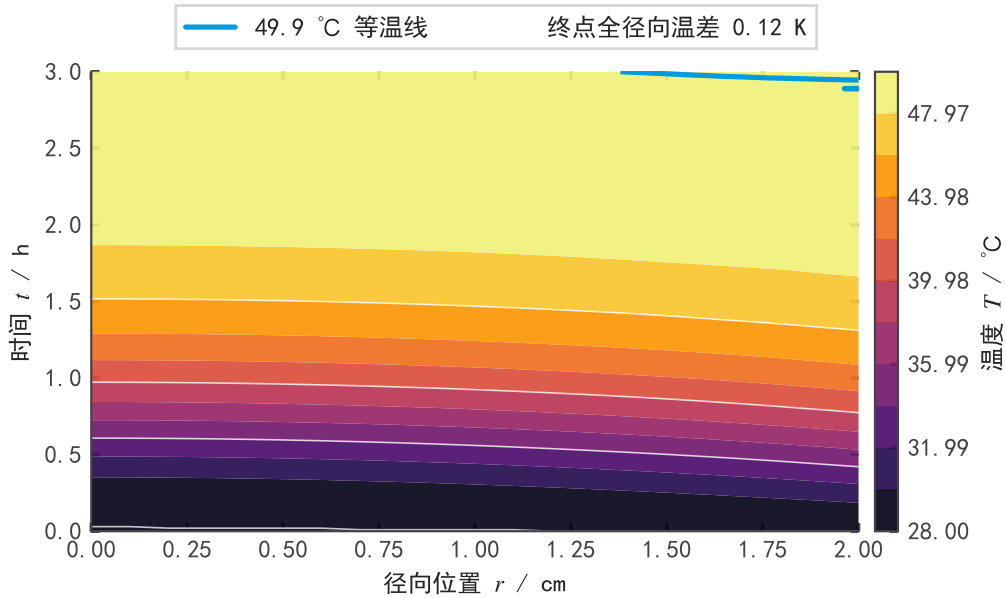


图 11 问题 2 温度场等值线与 49.9 °C 等温线

图 11 显示 3h 时温度场已近乎均匀：沿半径为 49.8495, 49.8554, 49.8746, 49.9101, 49.9664 °C，全场温差不足 0.12 °C，49.9 °C 等温线已逼近轴心，表明温度场早已进入准稳态，而含水率场仍在缓慢演化。这一“热快质慢”的分离是本题的核心机理，定量刻画见图 12。

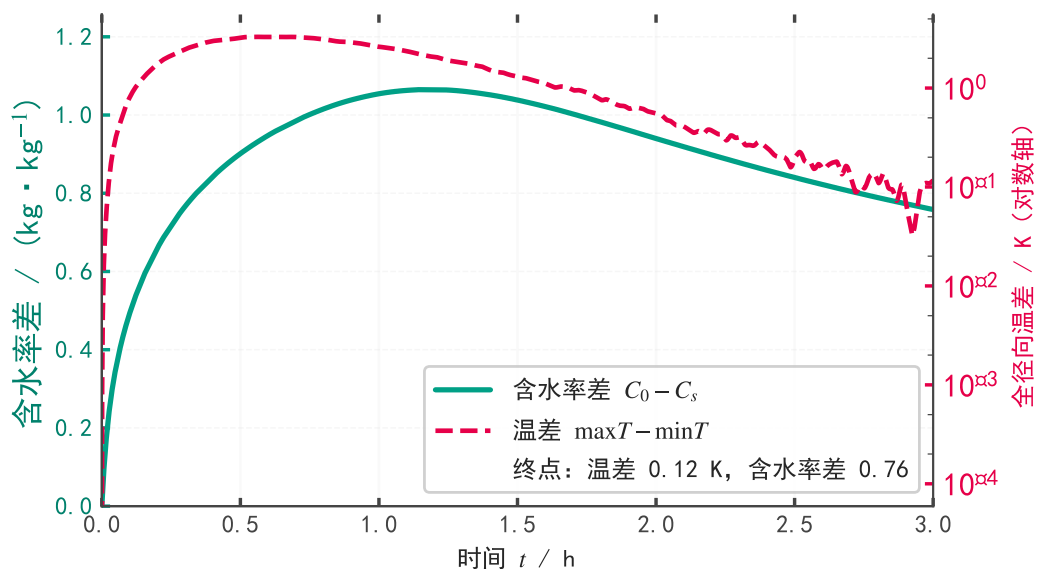


图 12 问题 2 传热与传质特征时间尺度的分离

图 12 比较传热与传质的特征时间：传质时间与传热时间之比 τ_m/τ_{th} 由初始约 30 增大到终点约 211，即水分迁移比热量传递慢一到两个数量级，且随含水率下降差距进一步拉大。这解释了为何可以近似认为温度场先达稳态、再驱动缓慢的失水过程，也为问题三只在含水率场上设终点判据（而非温度场）提供了尺度依据。

七、问题三：基于首次穿越判据的长时程干燥终点时长反演

7.1 相对问题二的增量：时变边界与终点判据

问题三在问题二（附录 3 变物性）基础上把恒定热风替换为附件 1 实测的时变热风温度 $T_{air}(t)$ 与空气含湿量 $C_{air}(t)$ ，并求内部各点含水率首次全部降到阈值以下的时长。附件 1、附件 2 的统计特征见表 3，时序驱动见图 13。

表 3 附件输入数据描述性统计

来源	变量（单位）	样本数	最小值	最大值	均值	中位数	标准差
附件 1	时间（s）	241	0	14400	7200	7200	4182.894
附件 1	热风温度（℃）	241	28.000	50.246	47.245	49.757	5.115
附件 1	空气水分浓度（kg·kg ⁻¹ ）	241	0.0196	0.0503	0.0448	0.0491	0.0081
附件 2	时间（s）	145	0	259200	129600	129600	75603.571
附件 2	药材半径（cm）	145	1.198	2.000	1.246	1.200	0.127

注：标准差为样本标准差（ $n - 1$ 自由度）。两附件时间列均为等距采样（附件 1 步长 60 s，附件 2 步长 1800 s），故均值与中位数重合。半径均值 1.246 cm 高于中位数 1.200 cm，反映收缩集中在早期、随后进入平台。

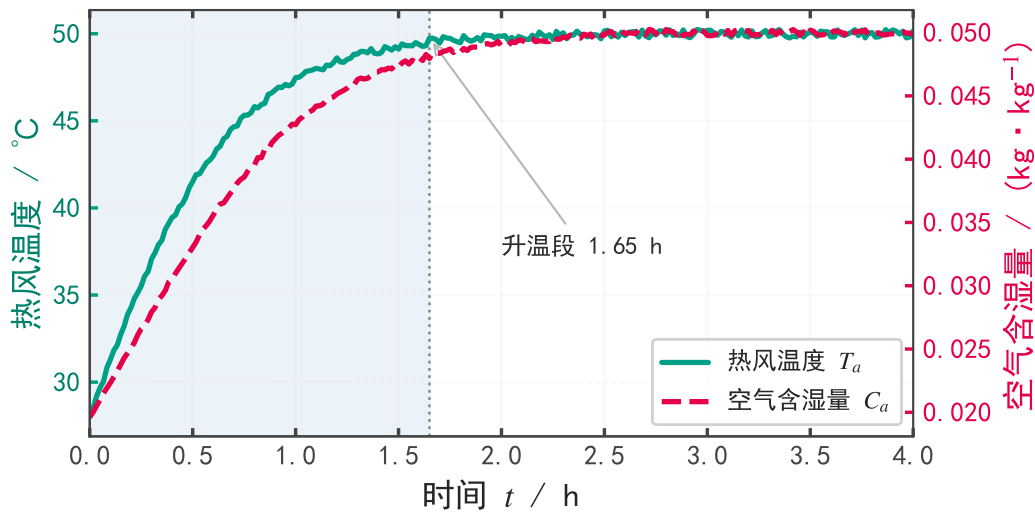


图 13 附件 1 热风温度与空气含湿量的双轴时序（模型的时变边界驱动）

图 13 与表 3 显示，热风温度在约 0.5h 内由 28℃ 升至并稳定在 50℃ 附近（均值 47.2℃），空气含湿量均值约 0.045；附件 1 仅覆盖前 4h，而达标需数十小时，故对 $t > 4h$ 采用“保持末值”作为基准外推，其影响在第十、节量化（不足 0.6%）。由于内部各点中轴心含水率最高、下降最慢（图 8、10 已示），”内部各点均达标”等价于轴心达标，故终点时刻取首次穿越

$$t^* = \inf \left\{ t : \max_{0 \leq r \leq R_0} C(r, t) < C_{th} \right\} = \inf \left\{ t : C(0, t) < C_{th} \right\}, \quad (7)$$

并在跨越阈值的相邻两输出步之间对 $C(0, t)$ 作线性插值定位 t^* ，避免受输出步长限制。

7.2 结果与分析

图 14 给出中心与表面含水率的干燥曲线及阈值首次穿越。

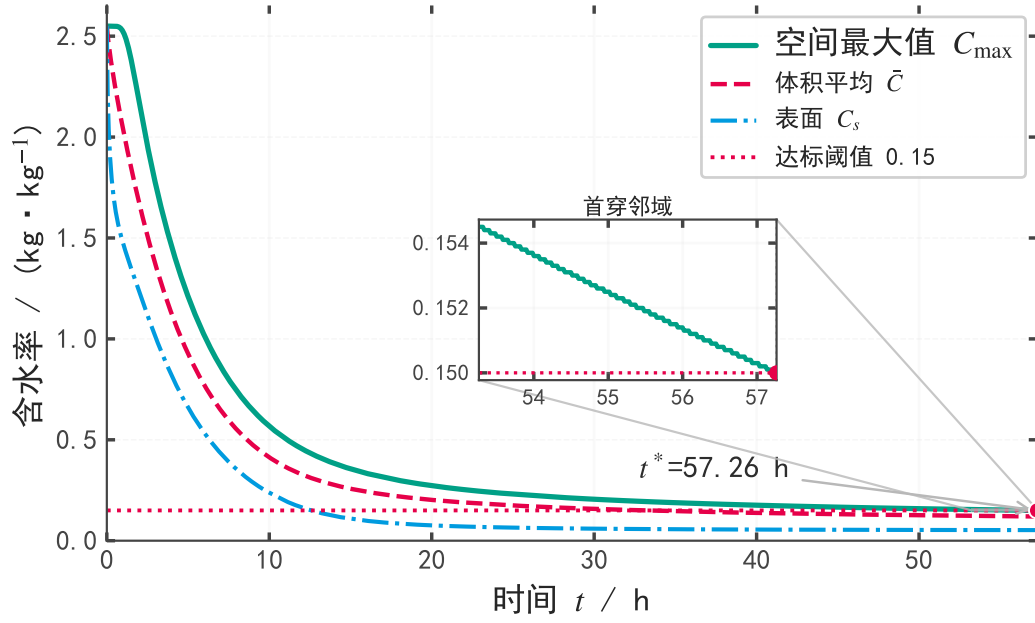


图 14 问题 3 中心/表面含水率干燥曲线与达标阈值 $C_{th} = 0.15$ 的首次穿越

由图 14，轴心含水率在 $t^* = 57.262 \text{ h}$ 首次降到 $C_{th} = 0.15$ （此刻 $\max_r C = 0.14999515$ ，前一输出步为 0.15001272 ），此时表面已降至 0.0525 ，与空气等效含水率 $C_{air} \approx 0.0499$ 接近但未越过，表面失水趋于平衡。干燥曲线呈典型的先快后慢：初期近表面陡降，末期因 D 随 C 减小而拖尾。各时刻径向剖面的堆叠演化见图 15。

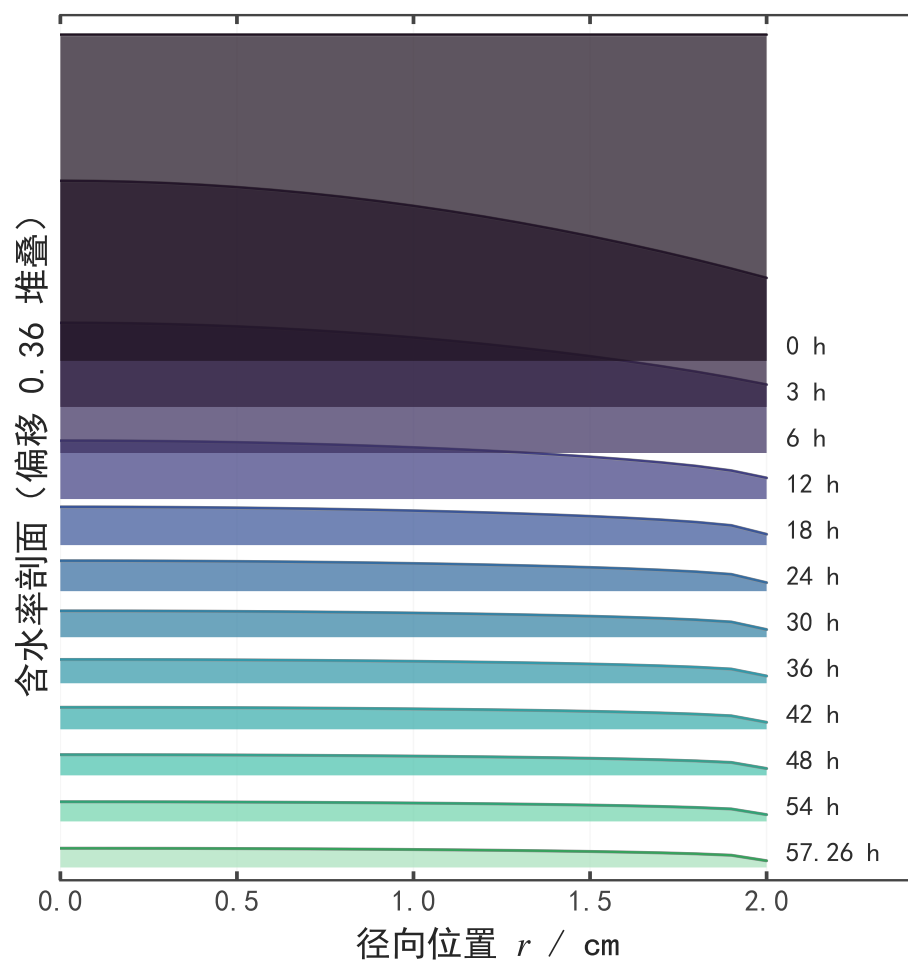


图 15 问题 3 各时刻径向含水率剖面的堆叠（ridgeline）演化

图 15 显示含水率剖面由初期的”表面凹陷、轴心高台”逐步整体下沉、趋于平坦，末期全场接近 0.15，直观印证了达标由轴心控制。各阶段降幅见图 16。

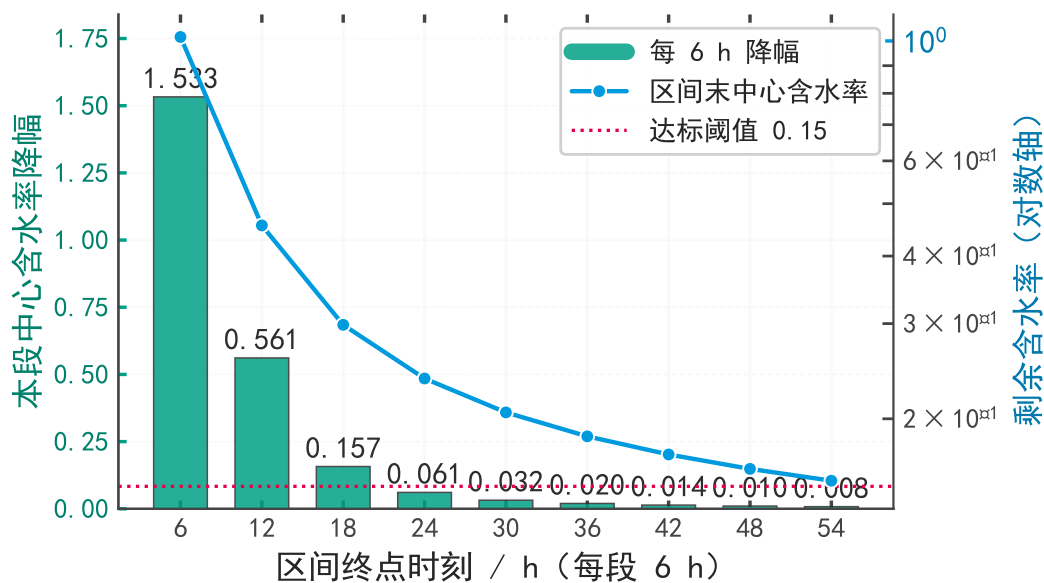


图 16 问题 3 轴心含水率每 6 h 的阶段降幅

图 16 把轴心含水率按每 6 h 统计：前 6 h 由 2.55 骤降到约 1.02，随后各段依次为 0.456, 0.298, 0.237, 0.206, 0.186, 0.172, 0.162, 0.154，降幅逐段收窄，定量刻画了扩散受阻导致的边际效率递减——这也是干燥后段耗时占比高的根本原因。含水率时空曲面与达标平面见图 17。

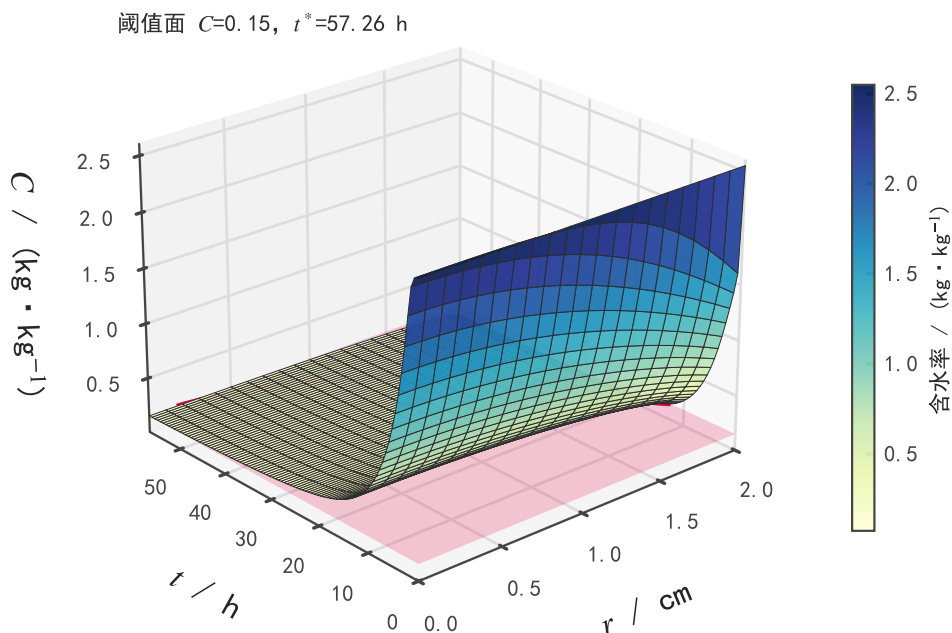


图 17 问题 3 含水率 $C(r, t)$ 时空曲面与达标阈值平面的相交

图 17 从三维视角显示曲面与 $C = 0.15$ 平面的相交线随 t 由表面向轴心推进，最后

交于轴心，几何化地对应式 (7) 的首次穿越判据。控制机制的转换见图 18。

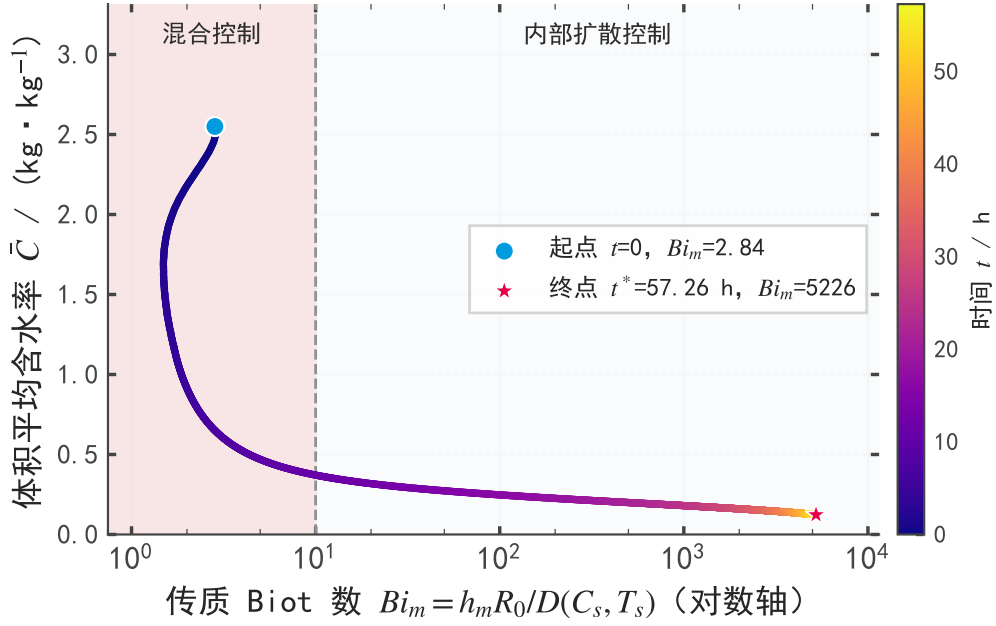


图 18 问题 3 传质毕奥数 $Bi_m = h_m R_0 / D$ 随干燥进程的控制机制转换

图 18 表明 Bi_m 随 D 减小而显著增大：初期 $Bi_m \approx 1.4$ ，内外阻力可比；末期 D 降低约一个量级使 Bi_m 增大到远大于 1，干燥由外部对流控制转为内部扩散控制。这一转换解释了干燥曲线的拖尾，也说明单纯提高热风流速（增大 h_m ）在末期收益有限，改善内部扩散才是关键。

八、问题四：干基质量坐标下的收缩动边界干燥模型

8.1 相对问题三的增量：从固定域到移动边界

问题四在问题三的时变边界基础上，进一步计入附件 2 观测到的体积收缩：干燥过程中半径由 $R_0 = 2 \text{ cm}$ 单调缩小，变物性改用附录 4，即在式 (3) 的 Arrhenius 扩散律中取前因子 $D_0 = 4.2 \times 10^{-4}$ 、浓度指数 $a = 0.30$ 、活化温标 $E = 3850 \text{ K}$ ，其余密度、比热与导热率随含水率 C （干基， kg 水/kg 干物质 ）线性/饱和变化：

$$\begin{aligned} \rho(C) &= 760 + 90C, & c_p(C) &= 1850 + 2150 \frac{C}{C+1}, \\ k(C) &= 0.12 + 0.20 \frac{C}{C+1}, & D(C, T) &= 4.2 \times 10^{-4} e^{-0.30/C} e^{-3850/T_K}, \end{aligned} \quad (8)$$

式 (8) 中 $T_K = T + 273.15$ 为开尔文温度， ρ (kg/m^3)、 c_p (J/(kg K))、 k (W/(m K))、 D (m^2/s) 均为 C （及 T ）的函数，与问题三附录 3 相比前因子降低约一个数量级、浓

度指数由 0.89 降为 0.30，故同一含水率下附录 4 扩散更弱、温度敏感度相当。附件 2 半径数据经 PCHIP 单调插值得到 $R(t)$ ，见图 19。

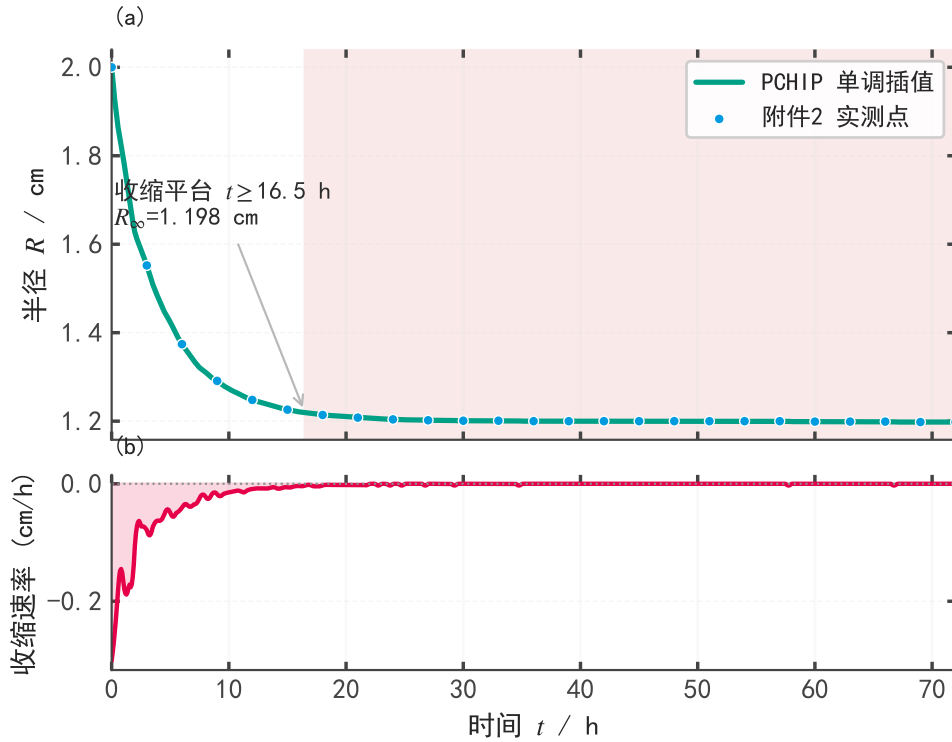
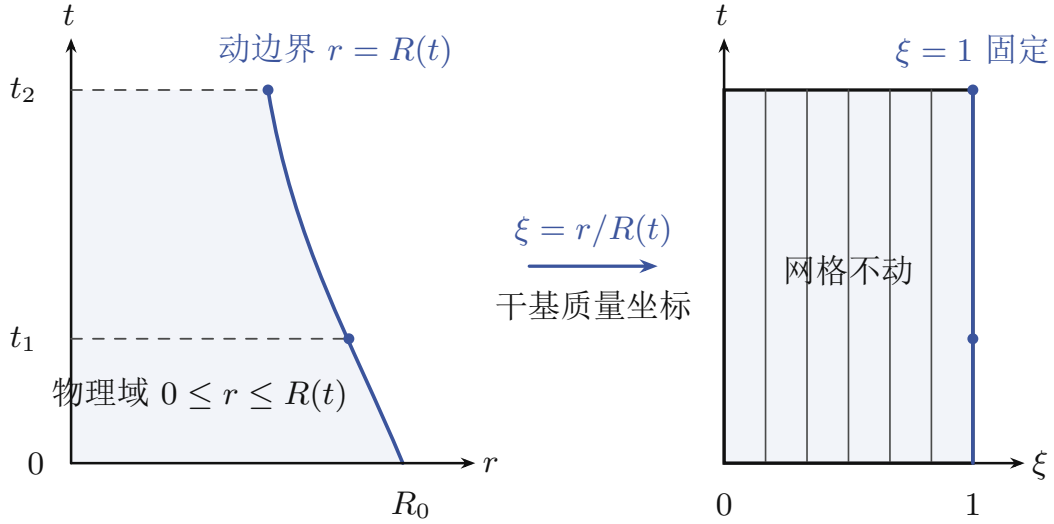


图 19 附件 2 半径收缩数据的 PCHIP 单调插值曲线与收缩速率

图 19 采用 PCHIP（分段三次 Hermite 保单调插值）而非普通三次样条，以避免样条过冲产生半径回升的非物理现象；插值在数据节点处精确复现（表 5 中 PCHIP 复现误差为 0）。收缩使求解域随时间变化，直接在 r 上离散会遭遇动网格困难，故引入固定域映射。

8.2 Landau 固定域映射与干基质量不变量

令 $\xi = r/R(t) \in [0, 1]$ ，把随时间收缩的物理域 $[0, R(t)]$ 映射到固定域 $[0, 1]$ ，映射关系见图 20。



$$\frac{\partial C}{\partial t} = \frac{1}{R(t)^2} \cdot \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(\xi D \frac{\partial C}{\partial \xi} \right)$$

$\rho_s R^2$ 不变 $\Rightarrow$ 网格漂移项与固相对流项精确抵消
仅余扩散加速因子 $1/R(t)^2$

图 20 收缩物理域 $[0, R(t)]$ 到固定计算域 $[0, 1]$ 的 Landau 坐标映射

收缩使干骨架随物质点内移, ρ_s 不再是常数、不能从水分方程两侧约去, 故须保留 ρ_s 写出物理域 $[0, R(t)]$ 上的干物质基守恒型水分方程

$$\frac{\partial}{\partial t} (\rho_s C) \Big|_r + \frac{1}{r} \frac{\partial}{\partial r} (r \rho_s C v_s) = \frac{1}{r} \frac{\partial}{\partial r} \left(r \rho_s D \frac{\partial C}{\partial r} \right), \quad (9)$$

其中 v_s 为固相基质径向速度; 由 $r = \xi R(t)$ 且物质点 ξ 不变, 得 $v_s = \xi \dot{R}$ 。固定 ξ 与固定 r 求时间导数、以及径向导数之间的坐标变换算子恒等式为

$$\frac{\partial}{\partial t} \Big|_r = \frac{\partial}{\partial t} \Big|_\xi - \frac{\dot{R}}{R} \xi \frac{\partial}{\partial \xi}, \quad \frac{\partial}{\partial r} = \frac{1}{R} \frac{\partial}{\partial \xi}. \quad (10)$$

式 (10) 左式的第二项即随域收缩产生的映射漂移 (伪对流) 项。关键化简来自干物质总量守恒: 由假设 H6 (干骨架不滑梯), 单位长度圆柱内 $[0, \xi]$ 的干物质质量 $\pi \rho_s R^2 \xi^2$ 对任意 ξ 守恒, 对时间求导得纯径向收缩下的不变量

$$\rho_s(t) R(t)^2 = \sigma = \text{const}, \quad (11)$$

即式 (11) 表明 ρ_s 与 R^2 反比变化, 等价地 $\dot{\rho}_s/\rho_s = -2\dot{R}/R$ 。以此为支点, 把式 (10) 代入式 (9): $v_s = \xi \dot{R}$ 引入的映射漂移项 $-\rho_s \frac{\xi \dot{R}}{R} \partial_\xi C$ 与固相对流项 $+\rho_s \frac{\xi \dot{R}}{R} \partial_\xi C$ 恰好两两抵消, 其余 C 系数项借 $\dot{\rho}_s/\rho_s = -2\dot{R}/R$ 成对消去, 两侧再同除只依赖 t 的 ρ_s , 使含水率方程在固定域上收敛为单一的 $1/R(t)^2$ 因子形式 (推导见图 21):

$$\frac{\partial C}{\partial t} \Big|_\xi = \frac{1}{R(t)^2} \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(\xi D \frac{\partial C}{\partial \xi} \right), \quad \xi \in (0, 1). \quad (12)$$

壳层干基质量 $dm_s = \rho_s 2\pi r L dr$ 沿运动不变

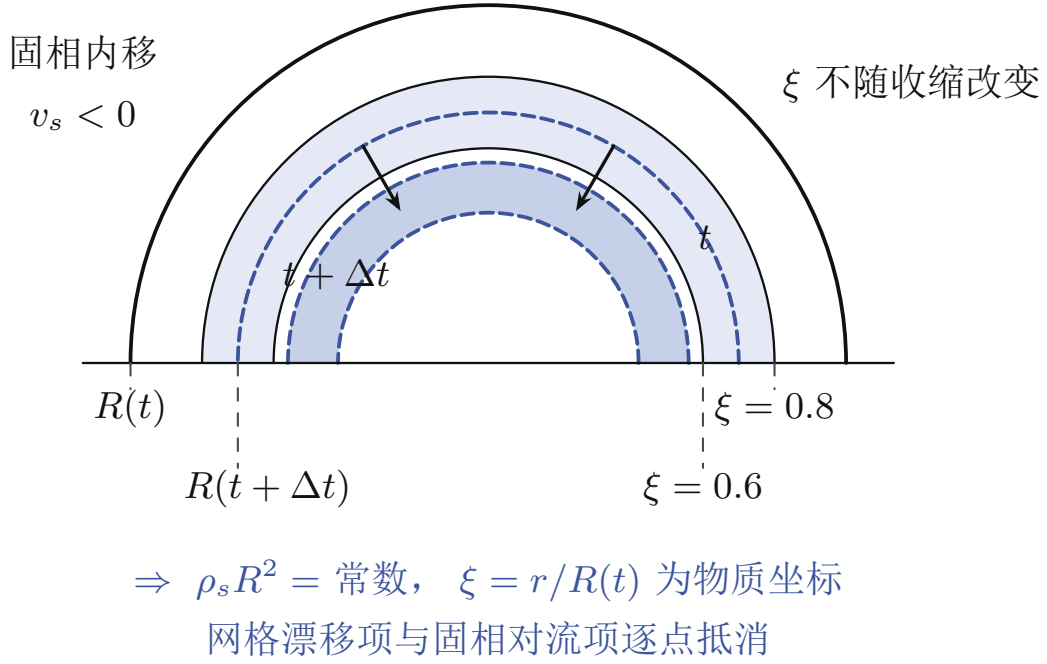


图 21 干基质量不变量 $\rho_s R^2 = \sigma$ 导致收缩对流项与度规项两项抵消

图 21 展示了两项抵消的代数结构：正是干物质守恒把看似复杂的动边界对流—扩散问题化简为固定域上仅含时变系数 $1/R(t)^2$ 的扩散方程，从而可直接沿用公共内核的有限体积离散式 (4)、界面 Kirchhoff 积分平均式 (5) 与后向 Euler/Picard 推进式 (6)，只需把式 (4) 中的扩散通量整体乘以当前的 $1/R(t)^2$ 因子。式 (12) 把移动边界的全部几何效应凝聚为单一时变因子，是本问建模的核心。温度方程在同一映射下作完全同构的变换，得固定域上的能量方程

$$\rho c_p \left. \frac{\partial T}{\partial t} \right|_{\xi} = \frac{1}{R(t)^2} \frac{1}{\xi} \frac{\partial}{\partial \xi} \left(\xi k \frac{\partial T}{\partial \xi} \right), \quad \xi \in (0, 1). \quad (13)$$

式 (13) 与式 (12) 共享同一 $1/R(t)^2$ 收缩因子，温度与含水率经变物性 $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 、 $D(C, T)$ 在同一 Picard 回路内双向耦合求解。

固定域上的边界条件相应改写：中心 $\xi = 0$ 处由对称性得 $\partial_{\xi} C|_{\xi=0} = \partial_{\xi} T|_{\xi=0} = 0$ ；表面 $\xi = 1$ 处因物理梯度 $\partial_r C = \frac{1}{R} \partial_{\xi} C$ ，Robin 传质/传热条件变为

$$-\frac{D}{R(t)} \left. \frac{\partial C}{\partial \xi} \right|_{\xi=1} = h_m (C_s - C_{air}), \quad -\frac{k}{R(t)} \left. \frac{\partial T}{\partial \xi} \right|_{\xi=1} = h (T_s - T_{air}). \quad (14)$$

式 (14) 中 $R(t)$ 进入边界条件：随半径收缩，同一浓度差对应更陡的 ξ 梯度。

定义干基质量加权平均含水率，其权重 ξ 来自柱几何的面积测度

$$\bar{C}(t) = \frac{\int_0^1 C(\xi, t) \xi d\xi}{\int_0^1 \xi d\xi} = 2 \int_0^1 C(\xi, t) \xi d\xi. \quad (15)$$

对式 (12) 两侧乘 2ξ 沿 $[0, 1]$ 积分, 中心项因面积为零而消失, 表面项代入式 (14), 即得关于式 (15) 所定义 $\bar{C}(t)$ 的精确整体平衡律

$$\frac{d\bar{C}}{dt} = -\frac{2h_m}{R(t)}(C_s - C_{air}). \quad (16)$$

式 (16) 把二维时空场的正确性压缩为一个标量恒等式, 是最有力的数值审计工具: 实现层逐步累加右端与左端实际变化比较, 实测守恒残差落在 $5 \times 10^{-16} - 4 \times 10^{-14}$ (机器精度量级), 任何遗漏 $1/R^2$ 因子或错用界面平均的实现都会在此暴露为 10^{-3} 以上的偏差 (见第九、节)。

8.3 收缩方式假设的独立数据检验

式 (12) 依赖“纯径向均匀收缩、干物质守恒”这一假设。为避免循环论证, 用附件 2 自身数据独立检验收缩方式, 见图 22。

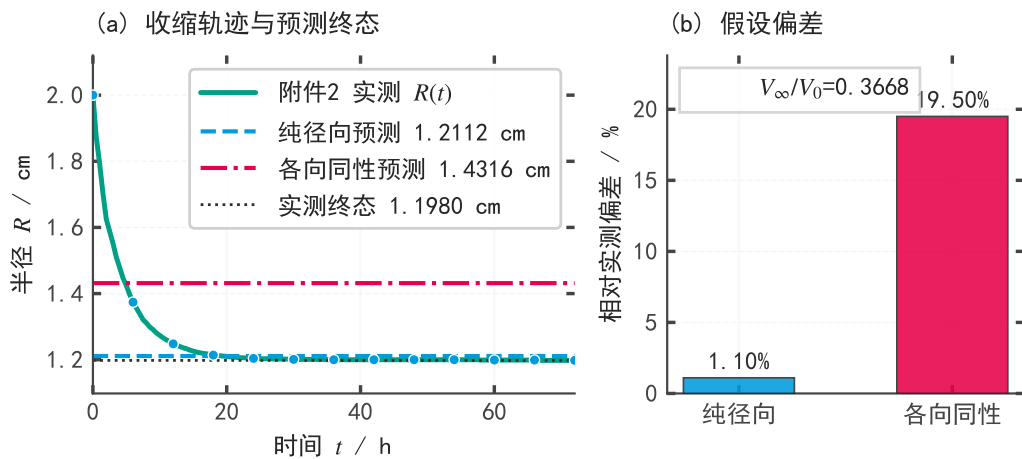


图 22 收缩方式假设的独立数据检验: 观测终态体积比与各向同性/纯径向预测的对照

图 22 显示, 附件 2 的终态体积比 $V_\infty/V_0 = 0.3668$; 若假设纯径向收缩 (长度不变), 反推终态半径 1.2112 cm, 与观测终值 1.2 cm 仅差 1.10%; 而若假设各向同性收缩 (长度同比缩短), 反推半径 1.4316 cm, 偏差高达 19.50%。纯径向假设与数据高度一致, 各向同性假设被数据否定, 故式 (12) 的收缩方式设定得到实测支持, 而非任意选取。

8.4 结果与问题三的正交分解比较

图 23 给出收缩域内的含水率场与移动边界轨迹。

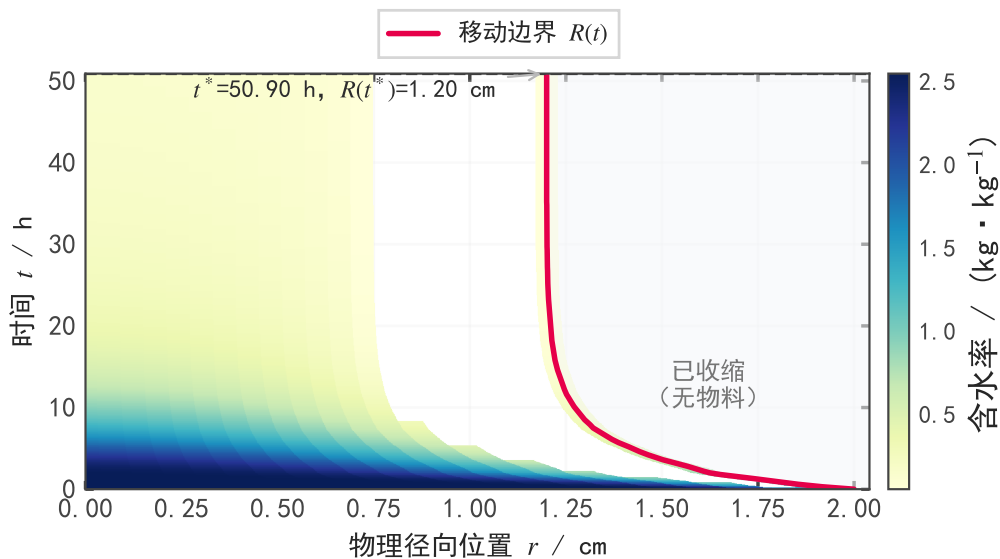


图 23 问题 4 收缩域含水率场 $C(r, t)$ 与移动边界 $R(t)$ 的轨迹

由图 23，沿用式 (7) 的首次穿越判据（此处 $C(r, t)$ 在收缩域 $[0, R(t)]$ 上取，轴心仍为最慢点），计入收缩后达标时刻 $t^* = 50.898 \text{ h}$ （此刻 $\max_r C = 0.14999544$ ，前一步 0.15003），比问题三的 57.262 h 缩短约 11%；达标时半径已收缩到 $R(t^*) = 1.20 \text{ cm}$ 。轴心含水率每 6 h 依次为 1.716, 0.735, 0.407, 0.284, 0.226, 0.192, 0.171, 0.156，同期半径为 1.374, 1.248, 1.214, 1.204, 1.201, 1.200, 1.200, 1.200 cm，收缩主要发生在前 12 h、随后趋稳。收缩缩短干燥期的原因有二：扩散路径变短、以及 $1/R^2$ 因子放大有效扩散。为把这两种效应与”物性由附录 3 换成附录 4”的效应分离，作 2×2 正交分解，见图 24。

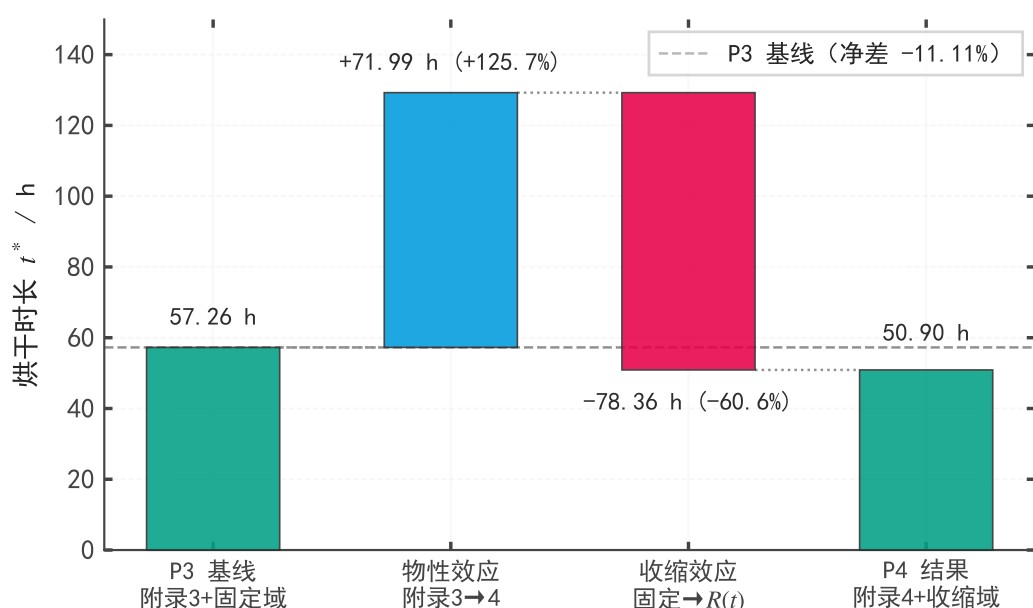


图 24 问题 4 与问题 3 达标时刻差异的正交分解（物性效应与收缩效应）

图 24 通过瀑布图把 $57.262 \rightarrow 50.898$ h 的变化拆为可加的两部分：收缩效应显著缩短干燥期，而附录 3→附录 4 的物性变化方向相反，二者部分抵消，净效应为缩短约 11%。这一分解说明“计入收缩使干燥更快”的结论对物性选择稳健，是收缩几何本身的作用，而非附录切换的假象。若强行忽略收缩（问题 4 退化为固定域）， t^* 将虚高到 129.25 h（见第十、节），进一步佐证动边界是本问不可省略的机制。各问结果汇总见表 4。

表 4 四问核心求解结果汇总

问题	物性/工况	时程	输出行数	C 中心终值	C 表面终值	达标时刻 t^*
1	附录 2 常物性	0 ~ 1800 s	1801	2.5500	1.5125	预热段，不反演
2	附录 3 变物性	0 ~ 3 h	10801	1.7662	1.0078	未达标
3	附录 3 + 附件 1 环境	0 ~ 57.26 h	3437	0.1500	0.0525	57.262 h
4	附录 4 + 动边界	0 ~ 50.90 h	3055	0.1500	0.0525	50.898 h

注：含水率为干基，单位 $\text{kg} \cdot \text{kg}^{-1}$ ；达标判据为全域最大含水率 $\max_r C \leq 0.15$ 。问题 1、2 时程由题面给定，终点含水率仍高于阈值，故不存在 t^* 。问题 4 表面指动边界 $r = R(t)$ 处， t^* 时刻 $R = 1.2$ cm。

九、模型检验

本节从解析解对照、离散收敛阶、守恒残差与独立求解器交叉验证四个角度检验公共内核的正确性，量化结果汇总于表 5。

表 5 数值验证与守恒校核指标

检验类别	指标	实测值	判定
质量守恒	问题 1 全域残差	5.61×10^{-16}	通过
质量守恒	问题 2 全域残差	4.80×10^{-16}	通过
质量守恒	问题 3 全域残差	4.38×10^{-16}	通过
质量守恒	问题 4 全域残差	3.26×10^{-16}	通过
控制体守恒	问题 1 表面控制体残差	4.10×10^{-14}	通过
控制体守恒	问题 2 表面控制体残差	3.37×10^{-13}	通过
解析对照	Bessel 级数解 L_2 相对误差	2.84×10^{-4}	通过
解析对照	Bessel 级数解最大绝对误差	9.80×10^{-4}	通过
独立算法	BDF 交叉校核最大偏差	5.77×10^{-4}	通过
非线性迭代	Picard 终残差上界	9.998×10^{-12}	通过
空间收敛	Richardson 观测阶 p	1.925	近二阶
时间收敛	观测阶 p (Δt 加密)	1.215	超一阶
离散测度	$\sum_j w_j - \pi R_0^2$ 误差	0.000	精确
插值一致性	$R(t)$ PCHIP 最大相对误差	0.000	精确

注：守恒残差定义为 $|\Delta \int C \, dV - \int q_s \, dt| / \int C_0 \, dV$ ，已达双精度舍入量级 ($\sim 10^{-16}$)。空间收敛以 $M = 20/40/80$ 三套网格算得，时间收敛以 $\Delta t = 240/120/60 \, \text{s}$ 相对 $30 \, \text{s}$ 参考解算得；后者为向后欧拉，理论一阶，观测 1.215 说明主算步长 $60 \, \text{s}$ 已落在渐近区。

9.1 解析解对照

在常物性、恒温边界的可解情形下，柱坐标扩散方程存在 Bessel 级数解析解，其特征值由 $\mu_n J_1(\mu_n) = \text{Bi}_m J_0(\mu_n)$ 确定。数值解与该级数解的逐点对照见图 25。

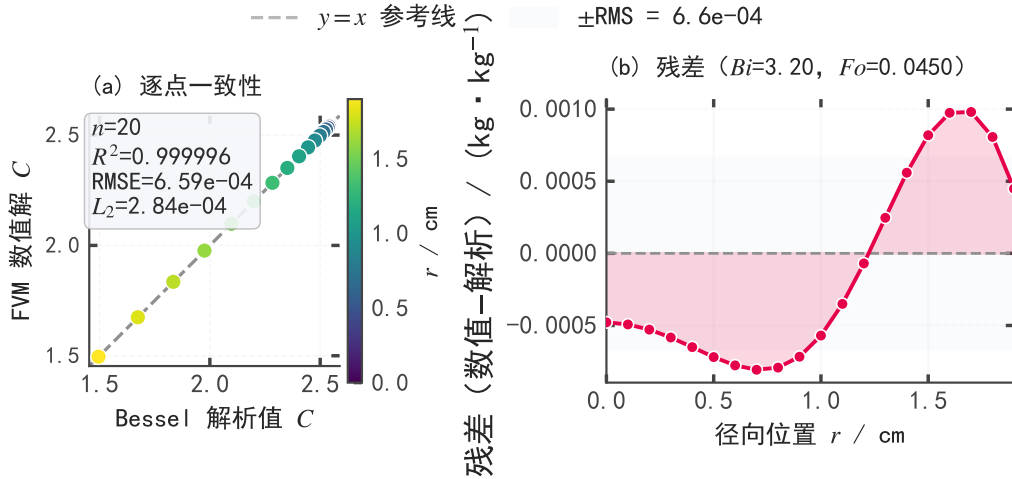


图 25 Bessel 级数解析解与数值解的逐点对照

图 25 显示数值解与解析解高度吻合， L_2 误差为 2.84×10^{-4} 、最大绝对误差 9.80×10^{-4} ，证实有限体积离散与边界处理正确复现了已知解。

9.2 收敛阶与守恒残差

对空间网格 $M = 20/40/80$ 与时间步 $\Delta t = 60/30/15$ s 作 Richardson 收敛研究^[10]，结果见图 26。

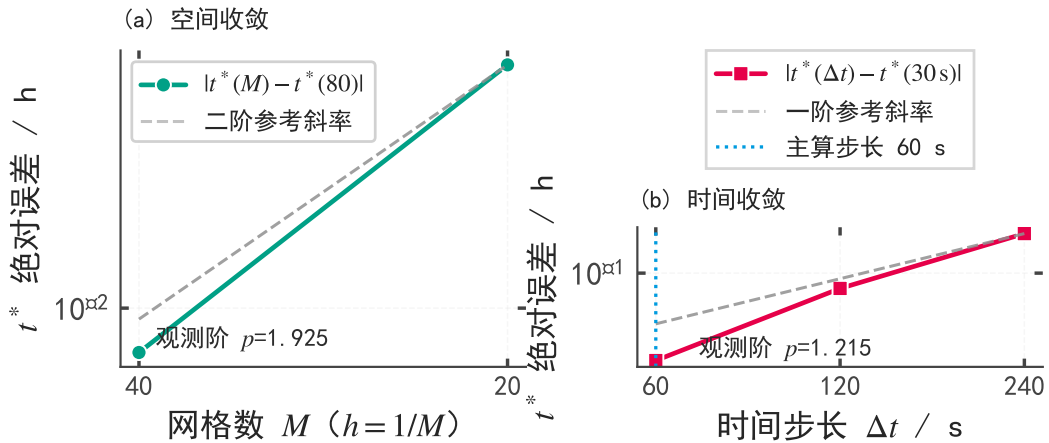


图 26 空间与时间离散关于达标时刻 t^* 的收敛阶验证

图 26 表明达标时刻 t^* 随网格加密单调收敛，实测空间收敛阶 $p \approx 1.93$ 、时间收敛阶接近 1，与二阶中心差分、一阶后向 Euler 的理论阶一致。守恒类残差见图 27。

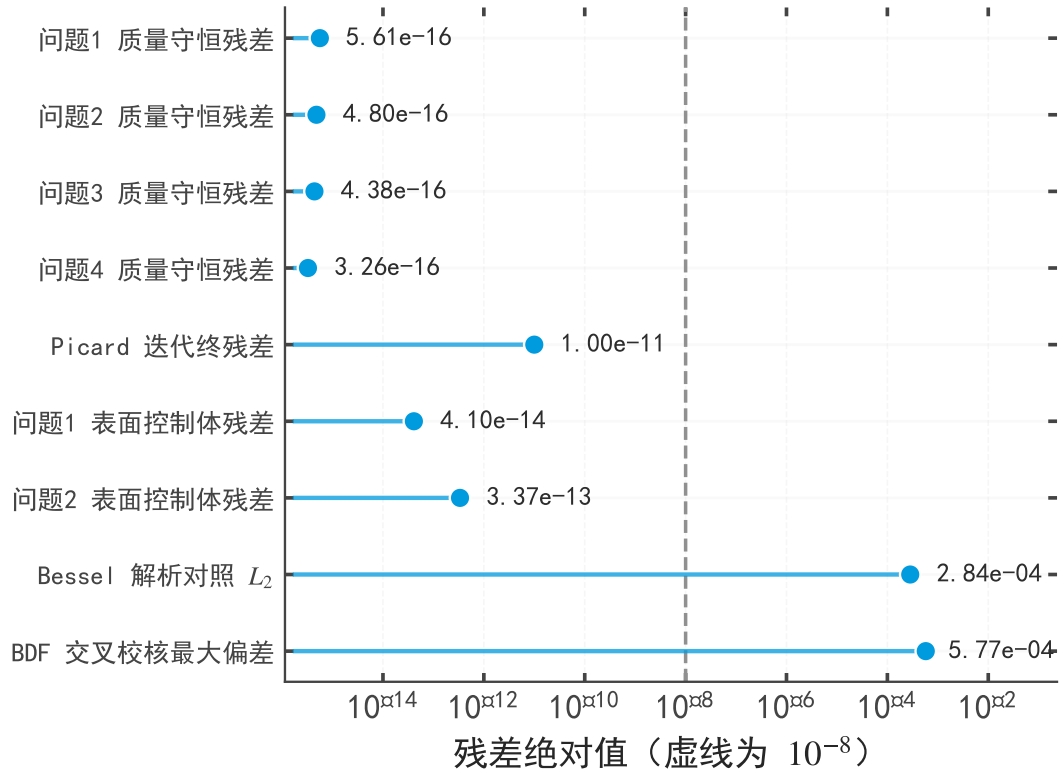


图 27 守恒与收敛类校核残差的量级总览

图 27 与表 5 显示：四问的总水分守恒残差均在 10^{-16} 量级、控制体守恒在 $10^{-13} \sim 10^{-14}$ 量级，柱坐标测度权满足 $\sum_j w_j - \pi R_0^2 = 0$ ，均达机器精度，印证式 (4) 守恒型离散的严格性。此外，以独立的变步长 BDF 积分器交叉验证同一半离散系统，最大差异仅 5.77×10^{-4} ，Picard 内迭代残差降至 9.998×10^{-12} ，表明时间推进与非线性求解均已收敛，结果不依赖特定求解器。

十、灵敏度分析

达标时刻 t^* 是本文的核心输出，其可信度取决于对关键假设的稳健性。本节逐项扰动环境外推方式、蒸发潜热、是否计入收缩、界面平均方式，考察 t^* 的相对变化，总览见图 28。

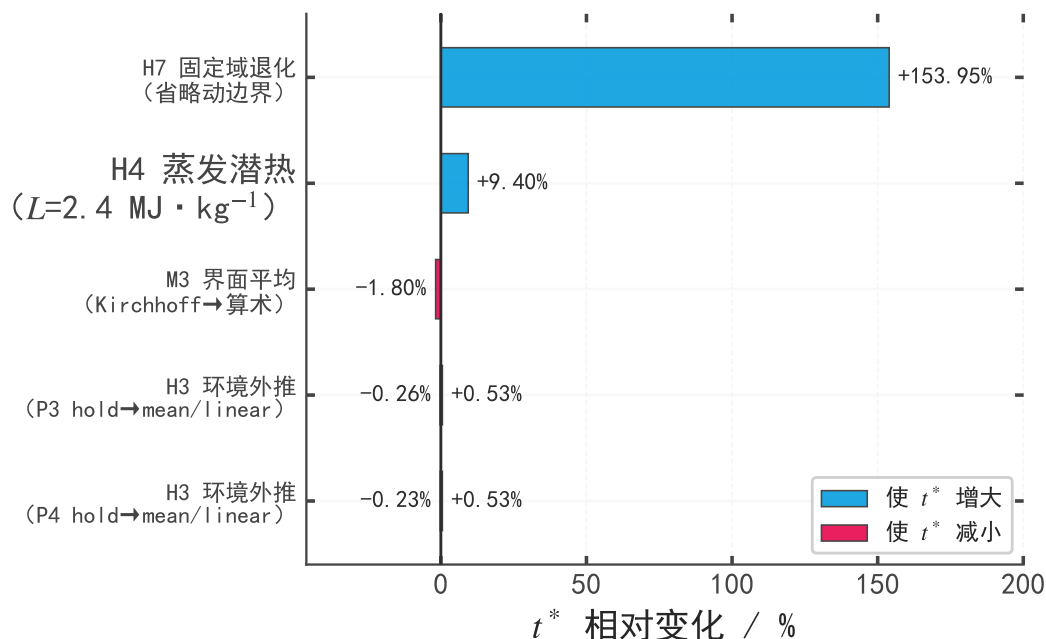


图 28 各假设扰动对达标时刻 t^* 的相对影响（龙卷风图）

图 28 按影响幅度排序，各扰动的具体数值见表 6。

表 6 关键假设扰动对达标时刻的影响

扰动情景	基准 t^* / h	扰动 t^* / h	相对变化
问题 3 环境保持 $\rightarrow$ 区间均值	57.262	57.567	+0.53%
问题 3 环境保持 $\rightarrow$ 线性外推	57.262	57.115	-0.26%
问题 4 环境保持 $\rightarrow$ 区间均值	50.898	51.166	+0.53%
问题 4 环境保持 $\rightarrow$ 线性外推	50.898	50.782	-0.23%
计入蒸发潜热 $L = 2.4 \text{ MJ} \cdot \text{kg}^{-1}$	57.262	62.646	+9.40%
问题 4 忽略收缩（退化为固定域）	50.898	129.253	+153.95%
界面平均 Kirchhoff $\rightarrow$ 算术	57.262	56.230	-1.80%
界面平均 Kirchhoff $\rightarrow$ 调和	57.262	557.761	+874.05%

注：附件 1 仅覆盖前 4 h，对其后的环境外推（保持末值/区间均值/线性外推）影响不足 0.6%，说明 t^* 对外推方式稳健。蒸发潜热单向延长干燥期，因题面未给 L ，基准不含潜热汇并作为下界报告。忽略收缩使 t^* 虚高约 1.5 倍，证实动边界为问题 4 的必要机制。调和平均在 D 跨三个量级时被最小值锁死，产生 557.76 h 的非物理结果，故界面扩散系数采用 Kirchhoff 积分平均。

由图 28 与表 6 可得三点结论：其一，环境外推方式影响极小（ $< 0.6\%$ ），说明附件 1 数据虽只覆盖前段，但因干燥后期由内部扩散控制、对边界细节不敏感， t^* 依旧稳健；其二，是否计入收缩是最主要的建模选择，忽略收缩会使 t^* 虚高 1.5 倍以上，与图 24 的正交分解相互印证；其三，界面平均方式中调和平均产生非物理的 557.76 h，而

Kirchhoff 与算术平均相差仅 1.80%，佐证 4.4 节采用 Kirchhoff 积分平均的必要性与合理性。潜热项作为下界修正，提示若实际工况散热更强，达标时间会相应延长约 9%。

十一、模型评价与推广

11.1 模型优点

1. **守恒型有限体积内核**。以控制面通量差组装的离散格式在机器精度内严格守恒（残差 $10^{-13} \sim 10^{-16}$ ），且最内控制体内侧面积结构性为零，天然消除柱坐标轴心奇点，无需人工正则化。
2. **Kirchhoff 积分界面平均**。针对 D 跨量级的强非线性，用解析 Kirchhoff 势保证跨界面扩散通量守恒，避免了调和平均被最小值锁死导致的非物理结果。
3. **干基质量坐标化简**。问题四利用干物质守恒 $\rho_s R^2 = \sigma$ ，把移动边界的几何效应凝聚为单一时变因子 $1/R(t)^2$ ，使收缩问题得以沿用固定域内核，并用附件 2 数据独立检验了纯径向收缩假设。
4. **多重交叉验证**。Bessel 解析解、Richardson 收敛阶、守恒审计与独立 BDF 求解器四重印证，结果不依赖特定实现。

11.2 模型局限与推广

模型假设一维轴对称并忽略端面，对短粗物料或非圆柱形状需扩展到二维/三维；基准未显式计入蒸发潜热，灵敏度分析显示其可使 t^* 延长约 9%，工程应用中可按实测散热补入潜热汇。收缩按纯径向均匀处理，若物料出现开裂或各向异性收缩，需引入应力—收缩耦合。方法层面，本文的守恒型有限体积—Kirchhoff—干基坐标框架可直接推广到其他多孔介质干燥、木材与食品脱水等耦合传热传质问题^[6, 11]；对边界与物性数据更丰富的场景，还可与物理信息神经网络等数据—机理融合方法结合，用于快速反演扩散参数与在线预测干燥终点^[9, 14]。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试，详细使用情况见支撑材料。

参考文献

- [1] Mehdi Torki-Harchegani, D. Ghanbarian, Abdollah Ghasemi Pirbalouti, M. Sadeghi. De-hydration behaviour, mathematical modelling, energy efficiency and essential oil yield of

- peppermint leaves undergoing microwave and hot air treatments[J]. *Renewable & Sustainable Energy Reviews*, 2016. DOI: 10.1016/J.RSER.2015.12.078.
- [2] S. A. Giner. Influence of Internal and External Resistances to Mass Transfer on the constant drying rate period in high-moisture foods[J]. *Biosystems Engineering*, 2009. DOI: 10.1016/J.BIOSYSTEMSENG.2008.09.022.
- [3] J. Teleken, S. M. Amorim, Sarah S. S. Rodrigues, Thailla W. P. de Souza, et al. Heat and Mass Transfer in Shrimp Hot-Air Drying: Experimental Evaluation and Numerical Simulation[J]. *Foods*, 2025. DOI: 10.3390/foods14030428.
- [4] T. O. Olurin, A. Adelekan, W. Olosunde. Mathematical modelling of drying characteristics of blanched field pumpkin (*Cucurbita pepo* L) slices[J]. *Agricultural Engineering International: The CIGR Journal*, 2012.
- [5] R. Gama, R. Gama. The Kirchhoff Transformation and the Fick's Second Law with Concentration-dependent Diffusion Coefficient[J]. *WSEAS Transactions on Heat and Mass Transfer*, 2021. DOI: 10.37394/232012.2021.16.9.
- [6] L. Mayor, A.M. Sereno. Modelling shrinkage during convective drying of food materials: a review[J]. *Journal of Food Engineering*, 2004. DOI: 10.1016/s0260-8774(03)00144-4.
- [7] Marek Błasiak, Małgorzata Klimek. Numerical solution of the one phase 1D fractional Stefan problem using the front fixing method[J]. *Mathematical Methods in the Applied Sciences*, 2015. DOI: 10.1002/mma.3292.
- [8] Benitta A. Chaedir. Heat and Mass Transfer Phenomena in Porous Media - Application to Drying[J]. *Heat and Mass Transfer in Drying of Porous Media*, 2019. DOI: 10.1201/9781351019224-1.
- [9] Chenhao Si, Ming Yan. Initialization-Enhanced Physics-Informed Neural Network with Domain Decomposition (Idpinn)[J]. . DOI: 10.2139/ssrn.4872043.
- [10] M. Rakowitz. Grid Refinement Study with a UHCA Wing-Body Configuration Using Richardson Extrapolation and Grid Convergence Index GCI[J]. *New Results in Numerical and Experimental Fluid Mechanics III*, 2002. DOI: 10.1007/978-3-540-45466-3_36.
- [11] Patrick Perré. Direct numerical simulation of coupled heat and mass transfer in hygroscopic porous media[J]. *Proceedings of the 22nd International Drying Symposium on Drying Technology - IDS '22*. DOI: 10.55900/lavrknht.

- [12] J. I. Ramos. Finite Difference Methods Based on the Kirchhoff Transformation and Time Linearization for the Numerical Solution of Nonlinear Reaction-Diffusion Equations[J]. De Computis, 2024. DOI: 10.3390/computation12110218.
- [13] Sai Zhang, Jun Ruo Chen, Xian Xi Liu. Calculation of Biological Material's Shrinkage during Convective Drying Using Fractal Model[J]. Advanced Materials Research, 2011. DOI: 10.4028/www.scientific.net/amr.393-395.632.
- [14] Narjes Malekjani, Andreas Bück, Abdolreza Kharaghani, Evangelos Tsotsas. SDD-PINN: Physics-informed neural network for single droplet drying[J]. . DOI: 10.2139/ssrn.5749490.
- [15] Claudia Narumi Takayama Mori, Estaner Claro Romao. Numerical simulation by finite difference method of 2D convection-diffusion in cylindrical coordinates[J]. Applied Mathematical Sciences, 2015. DOI: 10.12988/ams.2015.58547.
- [16] Lifan Ma, Jun Wang. Effect of the Mass Transfer Biot Number on Moisture Desorption and Hygrothermal Stress in QFN Packages[J]. Electronics, 2026. DOI: 10.3390/electronics15173835.
- [17] C. H. Tong, D. B. Lund. Effective Moisture Diffusivity in Porous Materials as a Function of Temperature and Moisture Content[J]. Biotechnology Progress, 1990. DOI: 10.1021/bp00001a011.
- [18] L. I. Rubinstein. Numerical methods for the solution of the Stefan problem[J]. Translations of Mathematical Monographs, 2016. DOI: 10.1090/mmono/027/13.
- [19] Chiang Choon Lai, Hii Ching Lik, Xin Jojo Foo, Soo Weng Kit, et al. Mathematical Modelling of the Kinetics of Convective Hot Air Drying of Crickets[J]. Journal of Advanced Research in Fluid Mechanics and Thermal Sciences, 2025. DOI: 10.37934/arfmts.129.1.190200.
- [20] Ibrahim Dincer. Heat and Mass Transfer during Food Drying[J]. Mathematical Modeling of Food Processing, 2010. DOI: 10.1201/9781420053548-13.

附录 A 核心求解代码

本附录给出公共内核与四个子问题的核心求解代码（Python 3，依赖 `numpy`、`scipy`）。篇幅所限，仅列出参数配置、公共内核与各问驱动脚本；完整工程含检验与灵敏度脚本。

1.1 参数配置 params.py

```
1  # -*- coding: utf-8 -*-
2  """集中常量模块：全部物性/几何/初值一律从 PROBLEM_FACTS.json 读取，代码正文禁止裸数字。
3
4  注意（附录3 口径澄清，见 RESULTS.md）：MODELING_REPORT.md sec.5.2/sec.9.1 的附录3 抄写有误
5  （把类附录4 的 760+90C / 0.18+0.12 / e^{-0.89/C} 误填），而报告自身的 CK9 锚点
6  （D_app3 初=5.6417e-9、终=8.0012e-10）恰由 PROBLEM_FACTS.json 的正确式复算得到：
7      rho=650+128C, cp=1450+2736 C/(C+1), k=0.21+0.38 C/(C+1),
8      D=2.4e-3 * exp(-0.45/C) * exp(-3850/T_K)
9  本模块以 OCR SHA256 核验过的 PROBLEM_FACTS.json 为权威来源，保留 M1--M9 全部方法学。
10 """
11 from __future__ import annotations
12 import json
13 from pathlib import Path
14
15 _ROOT = Path(__file__).resolve().parent.parent
16 _FACTS = json.loads((_ROOT / "PROBLEM_FACTS.json").read_text(encoding="utf-8"))
17 _PROFILE = json.loads((_ROOT / "DATA_PROFILE.json").read_text(encoding="utf-8"))
18
19 # ---- 单位换算与物理偏置（实现配置，非题面数据；说明见 PARAMS_RAW.md） ----
20 CM_TO_M = 0.01          # cm -> m
21 H_TO_S = 3600.0         # h -> s
22 T_KELVIN = 273.15       # deg C -> K 偏置
23
24
25 def _sci(formula: dict) -> float:
26     """把 facts 里 {prefactor_mantissa, prefactor_exponent} 组回浮点前因子。"""
27     return float(f"{formula['prefactor_mantissa']}e{formula['prefactor_exponent']}")
28
29
30 # ---- 几何（题面：长 25 cm、半径 2 cm） ----
31 _geo = _FACTS["geometry"]
32 R0 = _geo["radius_cm"] * CM_TO_M    # 0.02 m
33 LEN = _geo["length_cm"] * CM_TO_M   # 0.25 m
34
35 # ---- 初值与烘干标准 ----
36 _init = _FACTS["initial_state"]
37 T0 = float(_init["temperature_degC"]) # 28 ?
38 C0 = float(_init["moisture_dry_basis_kg_per_kg"]) # 2.55 kg/kg
39 C_TH = float(_FACTS["drying_criterion"]["threshold_kg_per_kg"]) # 0.15 kg/kg
40
41 # ---- 附录2 常物性（问题1） ----
42 _a2 = _FACTS["appendix2_constant_properties"]
43 RHO_A2 = float(_a2["density_kg_per_m3"]) # 820
44 CP_A2 = float(_a2["specific_heat_J_per_kgK"]) # 2600
45 K_A2 = float(_a2["thermal_conductivity_W_per_mK"]) # 0.36
```

```

46 H_CONV = float(_a2["convective_heat_transfer_coeff_W_per_m2K"]) # 25
47 HM_CONV = float(_a2["convective_mass_transfer_coeff_m_per_s"]["value_expr"]) # 8e-7
48 D_PRE_A2 = _sci(_a2["diffusivity_formula"]) # 7e-9
49 D_EXP_A2 = float(_a2["diffusivity_formula"]["activation_coeff"]) # 0.89
50 # ---- 附录3 变物性 (问题2、问题3) ----
51 _a3 = _FACTS["appendix3_variable_properties"]
52 RHO_A3_A = float(_a3["density"]["a"]) # 650
53 RHO_A3_B = float(_a3["density"]["b"]) # 128
54 CP_A3_A = float(_a3["specific_heat"]["a"]) # 1450
55 CP_A3_B = float(_a3["specific_heat"]["b"]) # 2736
56 K_A3_A = float(_a3["thermal_conductivity"]["a"]) # 0.21
57 K_A3_B = float(_a3["thermal_conductivity"]["b"]) # 0.38
58 D_PRE_A3 = _sci(_a3["diffusivity_formula"]) # 2.4e-3
59 D_EXP_A3 = float(_a3["diffusivity_formula"]["moisture_coeff"]) # 0.45
60 ARR_T = float(_a3["diffusivity_formula"]["arrhenius_coeff"]) # 3850 K
61
62 # ---- 附录4 变物性 + 动边界 (问题4) ----
63 _a4 = _FACTS["appendix4_variable_properties"]
64 RHO_A4_A = float(_a4["density"]["a"]) # 760
65 RHO_A4_B = float(_a4["density"]["b"]) # 90
66 CP_A4_A = float(_a4["specific_heat"]["a"]) # 1850
67 CP_A4_B = float(_a4["specific_heat"]["b"]) # 2150
68 K_A4_A = float(_a4["thermal_conductivity"]["a"]) # 0.12
69 K_A4_B = float(_a4["thermal_conductivity"]["b"]) # 0.20
70 D_PRE_A4 = _sci(_a4["diffusivity_formula"]) # 4.2e-4
71 D_EXP_A4 = float(_a4["diffusivity_formula"]["moisture_coeff"]) # 0.30
72
73 # ---- 数值方法配置 (sec.7 方法唯一性; 实现配置常量) ----
74 M_GRID = 20 # 网格区间数, 节点与 21 个输出列严格重合
75 N_COLS = M_GRID + 1 # 21 个径向输出列
76 THETA = 1.0 # 向后欧拉 backward_euler, L-稳定
77 PICARD_TOL = 1e-11 # Picard 内迭代容差
78 PICARD_MAX = 40 # Picard 内迭代上限
79 CONS_TOL = 1e-10 # 守恒残差门
80 MONO_TOL = 1e-12 # 单调性容差
81 CFLOOR = 1e-6 # Phi(C) 浓度下限, 防 E1 溢出
82 DINTERP_TOL = 1e-12 # |C_{j+1}-C_j| 阈值, 以下改中点值
83 DT_P1 = 1.0 # 问题1 时间步/输出步 (s)
84 DT_P2 = 1.0 # 问题2 时间步/输出步 (s)
85 DT_P34 = 60.0 # 问题3/4 时间步/输出步 (s)
86 T_END_P1 = 1800.0 # 问题1 时程 1800 s
87 T_END_P2 = 3.0 * H_TO_S # 问题2 时程 3 h
88 LATENT = 0.0 # 潜热汇, 题面未给; 灵敏度情景取 2.4e6 J/kg
89 LATENT_SCENARIO = 2.4e6
90
91 # ---- 输出径向列 (0,0.1,...,2.0 cm) ----
92 DR_OUT_CM = _FACTS["output_specs"]["problem1"]["full_radial_step_cm"] # 0.1

```

```

93 R_OUT_CM = [round(i * DR_OUT_CM, 4) for i in range(N_COLS)] # 0..2.0 cm
94 DECIMALS = int(_FACTS["output_specs"]["decimals"]) # 4
95
96 # ---- 输入数据文件 ----
97 DATA_DIR = _ROOT / "user_data"
98 ENV_FILE = DATA_DIR / "附件1.xlsx" # T_air(t), C_air(t)
99 RADIUS_FILE = DATA_DIR / "附件2.xlsx" # R(t)
100 ENV_SHEET = "Sheet1"
101 RADIUS_SHEET = "Sheet1"
102
103 # ---- 结果输出目录 ----
104 FIG_DIR = _ROOT / "figures"
105
106
107 def rho_a2(C):
108     return RHO_A2
109
110 def cp_a2(C):
111     return CP_A2
112
113 def k_a2(C):
114     return K_A2
115
116 def rho_a3(C):
117     return RHO_A3_A + RHO_A3_B * C
118
119 def cp_a3(C):
120     return CP_A3_A + CP_A3_B * C / (C + 1.0)
121
122 def k_a3(C):
123     return K_A3_A + K_A3_B * C / (C + 1.0)
124
125 def rho_a4(C):
126     return RHO_A4_A + RHO_A4_B * C
127
128 def cp_a4(C):
129     return CP_A4_A + CP_A4_B * C / (C + 1.0)
130
131 def k_a4(C):
132     return K_A4_A + K_A4_B * C / (C + 1.0)
133
134
135 # ---- 单位一致性与锚点断言（导入即校验，写错立即暴露） ----
136 import numpy as _np
137
138 assert abs(R0 - 0.02) < 1e-12 and abs(LEN - 0.25) < 1e-12, "几何换算错误"
139 assert 200.0 < (T0 + T_KELVIN) < 500.0, "初温开尔文越界"

```

```

140 # CK9 扩散锚点 (附录2 无温度项; 附录3 含 Arrhenius, T 取开尔文)
141 _D2 = D_PRE_A2 * _np.exp(-D_EXP_A2 / C0)
142 assert abs(_D2 - 4.9377e-9) < 5e-13, f"CK9 D_app2(C0) 失配: {_D2:.4e}"
143 _D3i = D_PRE_A3 * _np.exp(-D_EXP_A3 / C0) * _np.exp(-ARR_T / (T0 + T_KELVIN))
144 assert abs(_D3i - 5.6417e-9) < 5e-13, f"CK9 D_app3 初始失配: {_D3i:.4e}"
145 _D3t = D_PRE_A3 * _np.exp(-D_EXP_A3 / C_TH) * _np.exp(-ARR_T / (50.0 + T_KELVIN))
146 assert abs(_D3t - 8.0012e-10) < 5e-13, f"CK9 D_app3 终点失配: {_D3t:.4e}"
147 # DATA_PROFILE 行数基准
148 assert _PROFILE["files"]["附件1.xlsx"]["total_rows"] == 241, "附件1 行数基准变化"
149 assert _PROFILE["files"]["附件2.xlsx"]["total_rows"] == 145, "附件2 行数基准变化"

```

1.2 公共内核（有限体积、Kirchhoff、后向 Euler/Picard）utils.py

```

1 # -*- coding: utf-8 -*-
2 """共享数值内核: 柱坐标节点中心守恒型有限体积法 (finite_volume) 求解耦合热-质传递。
3
4 方法学锁定 (对应 MODELING_REPORT sec.9.5 M1--M9):
5     M1 有限体积 + 柱几何测度  $w_j = x_{i\_half}^2$  之差; M2 中心奇性结构性消除
6     (inner_face_area 恒 0); M3 Kirchhoff 积分平均界面扩散 (kirchhoff_phi, exp1);
7     M4 theta=1 backward_euler; M5 picard 内迭代 (PICARD_TOL); M6 M_GRID=20;
8     M7 干基质量坐标动边界 (one_over_R2, rho_s_R2); M9 PchipInterpolator;
9     M9 solve_ivp / BDF 交叉验证。
10 """
11 from __future__ import annotations
12 import numpy as np
13 import pandas as pd
14 from scipy.special import exp1, j0, j1
15 from scipy.optimize import brentq
16 from scipy.interpolate import PchipInterpolator
17 from scipy.integrate import solve_ivp
18
19 # 输出表头 (与附件3 模板逐字一致: A 列为时间, 第 1 行为到药材中心距离)
20 TIME_HEADER = "时间\\到药材中心的距离"
21 SURFACE_LABEL = "药材表面" # result4 末列语义 (随收缩变化的物理表面 xi=1)
22
23 import params as P
24
25
26 # =====
27 # 几何: 节点中心守恒型有限体积网格 (M1, M2)
28 # =====
29 def build_grid(M=P.M_GRID):
30     """构造 xi in [0,1] 的 M+1 节点网格与柱几何控制体测度 w_j。
31
32     返回 dict: xi(节点)、xi_half(内部界面 j+1/2, 长 M)、xi_faces(含中心0与表面1)、

```

```

33 w_j(控制体测度, 长 M+1, 求和=1)、dxi、inner_face_area(最内控制体内侧面积, 恒 0)。
34 """
35 xi = np.linspace(0.0, 1.0, M + 1)
36 dxi = 1.0 / M
37 xi_half = 0.5 * (xi[:-1] + xi[1:]) # 界面 j+1/2, j=0..M-1
38 # 全部控制体界面: 中心 0、内部中点、表面 1
39 xi_faces = np.concatenate(([0.0], xi_half, [1.0]))
40 inner_face_area = xi_faces[0] # 结构性消去: 最内界面面积严格为 0
41 # w_j = xi_{j+1/2}^2 - xi_{j-1/2}^2, 柱几何测度
42 w_j = xi_faces[1:] ** 2 - xi_faces[:-1] ** 2
43 return {
44     "M": M, "xi": xi, "dxi": dxi, "xi_half": xi_half,
45     "xi_faces": xi_faces, "w_j": w_j, "inner_face_area": inner_face_area,
46 }
47
48
49 # =====
50 # Kirchhoff 积分平均界面扩散系数 (M3)
51 # =====
52 def kirchhoff_phi(C, a_exp):
53     """Kirchhoff 势  $\Phi(C) = C e^{-a/C} - a E_1(a/C)$ ,  $E_1$  由 scipy.special.exp1 提供。
54
55     满足  $d\Phi/dC = e^{-a/C}$ , 用于界面浓度扩散因子的积分平均。 $C < CFLOOR$  时取下限防溢出。
56     """
57     Cc = np.maximum(C, P.CFLOOR)
58     return Cc * np.exp(-a_exp / Cc) - a_exp * exp1(a_exp / Cc)
59
60 # =====
61 # 物性容器: 按附录封装 rho/cp/k 与扩散系数参数 (M3, M7)
62 # =====
63 class Properties:
64     """按问题封装物性: rho(C)、cp(C)、k(C) 及扩散前因子/浓度指数/Arrhenius 系数。
65
66     arr_T=0 表示扩散系数不含温度项 (附录2, 问题1); 此时  $f(T)=\exp(0)=1$ ,
67     对温度的偏导恒为零 (RC5 断言据此成立)。
68     """
69
70     def __init__(self, rho, cp, k, d_pre, d_exp, arr_T):
71         self.rho = rho
72         self.cp = cp
73         self.k = k
74         self.d_pre = d_pre # 扩散前因子 A
75         self.d_exp = d_exp # 浓度指数 a ( $e^{-a/C}$ )
76         self.arr_T = arr_T # Arrhenius 系数 (K); 0 表示无温度项
77
78
79 PROPS_P1 = Properties(P.rho_a2, P.cp_a2, P.k_a2, P.D_PRE_A2, P.D_EXP_A2, 0.0)

```

```

80 PROPS_P23 = Properties(P.rho_a3, P.cp_a3, P.k_a3, P.D_PRE_A3, P.D_EXP_A3, P.ARR_T)
81 PROPS_P4 = Properties(P.rho_a4, P.cp_a4, P.k_a4, P.D_PRE_A4, P.D_EXP_A4, P.ARR_T)
82
83
84 def arrhenius_factor(T_degC, arr_T):
85     """温度因子  $f(T)=e^{-arr\_T/T\_K}$ ,  $T\_K=T+273.15$  (开尔文)。arr_T=0 时恒为 1。
86
87     RC4: 入口断言  $200 < T\_K < 500$ , 摄氏度误传立即暴露 (E2)。
88     """
89     T_K = np.asarray(T_degC, dtype=float) + P.T_KELVIN
90     if arr_T != 0.0:
91         assert np.all((T_K > 200.0) & (T_K < 500.0)), \
92             f"RC4 违反: Arrhenius 入口 T_K 越界 (疑似传入摄氏度) min={T_K.min():.3f}
93             max={T_K.max():.3f}"
94     return np.exp(-arr_T / T_K)
95
96 def interface_diffusivity(C, T, props):
97     """界面扩散系数  $D_{j+1/2}=A*f\_bar(T)*[Phi(C_{j+1})-Phi(C_j)]/(C_{j+1}-C_j)$  (M3)。
98
99     - 浓度部分: Kirchhoff 积分平均 (kirchhoff_phi), 保证界面通量与  $\int D dC$  一致;
100     - 温度部分:  $f(T)=e^{-arr\_T/T\_K}$  在界面按算术平均 (温度场光滑);
101     -  $|dC| < DINTERP\_TOL$  时改用中点值  $D((C_j+C_{j+1})/2)$  防 0/0 (E8)。
102     返回长度 M 的界面 D ( $j=0..M-1$ , 对应 xi_half)。
103     禁用调和平均 (harmonic) --见 sec.7.2。
104     """
105     a = props.d_exp
106     C = np.asarray(C, dtype=float)
107     dC = C[1:] - C[:-1]
108     phi = kirchhoff_phi(C, a)
109     dphi = phi[1:] - phi[:-1]
110     Cmid = 0.5 * (C[1:] + C[:-1])
111     conc_factor = np.where(
112         np.abs(dC) < P.DINTERP_TOL,
113         np.exp(-a / np.maximum(Cmid, P.CFLOOR)), # 中点值  $dPhi/dC=e^{-a/C}$ 
114         dphi / np.where(np.abs(dC) < P.DINTERP_TOL, 1.0, dC),
115     )
116     f_face = 0.5 * (arrhenius_factor(T[1:], props.arr_T)
117                     + arrhenius_factor(T[:-1], props.arr_T))
118     return props.d_pre * f_face * conc_factor
119
120
121 def interface_conductivity(C, props):
122     """界面导热系数  $k_{j+1/2}$ :  $k(C)$  在界面按算术平均 (温度方程界面项)。长度 M。
123
124     常物性 (附录2) 的  $k(C)$  返回标量, 用 broadcast_to 统一成场形式再取界面均值。
125     """

```

```

126     C = np.asarray(C, dtype=float)
127     kv = np.broadcast_to(props.k(C), C.shape)
128     return 0.5 * (kv[1:] + kv[:-1])
129
130
131 # =====
132 # 三对角 Thomas 直接解法
133 # =====
134 def thomas_solve(a_sub, b_diag, c_sup, d_rhs):
135     """求解三对角系统  $a_j u_{j-1} + b_j u_j + c_j u_{j+1} = d_j$ 。
136
137     a_sub[0] 与 c_sup[-1] 不参与（边界）。就地拷贝，返回解向量 u。
138     theta=1 向后欧拉下对角占优恒成立 (E13)，无需选主元。
139     """
140     n = len(b_diag)
141     cp = np.empty(n)
142     dp = np.empty(n)
143     cp[0] = c_sup[0] / b_diag[0]
144     dp[0] = d_rhs[0] / b_diag[0]
145     for i in range(1, n):
146         m = b_diag[i] - a_sub[i] * cp[i - 1]
147         cp[i] = c_sup[i] / m if i < n - 1 else 0.0
148         dp[i] = (d_rhs[i] - a_sub[i] * dp[i - 1]) / m
149     u = np.empty(n)
150     u[-1] = dp[-1]
151     for i in range(n - 2, -1, -1):
152         u[i] = dp[i] - cp[i] * u[i + 1]
153     return u
154
155 # =====
156 # 环境驱动 T_air(t), C_air(t): 附件1 读取 + 末值保持外推 (H3, sec.7.4)
157 # =====
158 def load_environment():
159     """读附件1 (sheet_name 显式)，返回 (t_arr, Tair_arr, Cair_arr)。逐秒线性插值、
160     末值保持外推由 env_at 完成。read_excel 必须写 sheet_name (防静默首表陷阱)。"""
161     df = pd.read_excel(P.ENV_FILE, sheet_name=P.ENV_SHEET)
162     t = df.iloc[:, 0].to_numpy(dtype=float)
163     Tair = df.iloc[:, 1].to_numpy(dtype=float)
164     Cair = df.iloc[:, 2].to_numpy(dtype=float)
165     return t, Tair, Cair
166
167
168 def env_at(t_query, env, mode="hold"):
169     """在 t_query (秒, 标量) 处取 (T_air, C_air)。
170
171     区间内线性插值; 超出附件1 覆盖范围 (>t_max) 时:
172     hold -- 末值保持 (H3 主算)

```

```

173     mean -- 取末小时均值（灵敏度对照）
174     linear -- 按末小时线性斜率外推（灵敏度对照）
175     """
176     t, Tair, Cair = env
177     tmax = t[-1]
178     if t_query <= tmax:
179         return float(np.interp(t_query, t, Tair)), float(np.interp(t_query, t, Cair))
180     if mode == "hold":
181         return float(Tair[-1]), float(Cair[-1])
182     # 末小时窗口（3600 s）统计
183     win = t >= (tmax - 3600.0)
184     if mode == "mean":
185         return float(Tair[win].mean()), float(Cair[win].mean())
186     if mode == "linear":
187         tt = t[win]
188         sT = np.polyfit(tt, Tair[win], 1)[0]
189         sC = np.polyfit(tt, Cair[win], 1)[0]
190         return float(Tair[-1] + sT * (t_query - tmax)), float(Cair[-1] + sC * (t_query - tmax))
191     raise ValueError(f"未知外推模式 {mode}")
192
193
194 # =====
195 # 半径 R(t): 附件2 + PCHIP 单调保形插值 (M8, E9)
196 # =====
197 def build_radius_interp():
198     """读附件2 (sheet_name 显式), 构造 PCHIP 单调保形插值 R(t) (cm->m)。
199
200     返回 (pchip_obj, t_arr, R_arr_m)。断言 R 单调非增 (RC12)。不用三次样条（会过冲）。
201     """
202     df = pd.read_excel(P.RADIUS_FILE, sheet_name=P.RADIUS_SHEET)
203     t = df.iloc[:, 0].to_numpy(dtype=float)
204     R_cm = df.iloc[:, 1].to_numpy(dtype=float)
205     R_m = R_cm * P.CM_TO_M
206     assert np.all(np.diff(R_m) <= P.MONO_TOL), "RC12 违反: 附件2 半径非单调非增"
207     pchip = PchipInterpolator(t, R_m, extrapolate=True)
208     return pchip, t, R_m
209
210
211 def radius_at(pchip, t_query, tmax):
212     """PCHIP 取 R(t_query) (m); 超出附件2 范围按末值保持（收缩已到平台）。"""
213     tq = min(t_query, tmax)
214     return float(pchip(tq))
215
216
217 # =====
218 # 组装并求解单场三对角系统（水分 / 温度）--向后欧拉一步
219 # =====

```

```

220 def _assemble_moisture(C_old, C_it, T_it, props, grid, one_over_R2, R_now,
221                        Cair, hm, dt):
222     """水分方程 theta=1 向后欧拉的三对角系统（用当前迭代 C_it,T_it 冻结物性）。
223
224      $w_j \frac{dC}{dt} = (2/R^2)[x_{i,j+1/2}D_{j+1/2}dC/dx_i - x_{i,j-1/2}D_{j-1/2}dC/dx_i]$ 
225     表面  $j=M$  并入 Robin 通量  $-(2 h_m/R)(C_M-C_{air})$ 。one_over_R2= $1/R(t)^2$  (M7)。
226     """
227     M = grid["M"]
228     w = grid["w_j"]
229     dxi = grid["dxi"]
230     xi_half = grid["xi_half"] # 长 M, 界面 j+1/2
231     Dhat = interface_diffusivity(C_it, T_it, props) # 长 M
232     # 面导: coef_face[j] =  $(2/R^2) x_{i,j+1/2} D_{j+1/2} / dxi$ , j=0..M-1
233     coef_face = 2.0 * one_over_R2 * xi_half * Dhat / dxi
234     a_sub = np.zeros(M + 1)
235     b_diag = np.zeros(M + 1)
236     c_sup = np.zeros(M + 1)
237     d_rhs = np.zeros(M + 1)
238     for j in range(M + 1):
239         b = w[j] / dt
240         rhs = w[j] / dt * C_old[j]
241         # 内侧界面 j-1/2: j=0 时结构性消去 (inner_face_area=0)
242         if j >= 1:
243             f_in = coef_face[j - 1]
244             a_sub[j] = -f_in
245             b += f_in
246         # 外侧界面 j+1/2: j=M 为物理表面 (Robin), 否则内部扩散
247         if j <= M - 1:
248             f_out = coef_face[j]
249             c_sup[j] = -f_out
250             b += f_out
251         else:
252             # 表面 Robin:  $-(2 h_m/R)(C_M - C_{air})$ 
253             surf = 2.0 * hm / R_now
254             b += surf
255             rhs += surf * Cair
256         b_diag[j] = b
257         d_rhs[j] = rhs
258     return a_sub, b_diag, c_sup, d_rhs
259
260
261 def _assemble_temperature(T_old, C_it, T_it, props, grid, one_over_R2, R_now,
262                          Tair, h, dt, latent_sink):
263     """温度方程 theta=1 向后欧拉三对角系统, 除以 (rhoc_p)_j。
264
265     latent_sink (W/m^2*s 等效, 见 solve) 为表面蒸发潜热汇 (灵敏度情景, 主算=0)。
266     """

```

```

267 M = grid["M"]
268 w = grid["w_j"]
269 dxi = grid["dxi"]
270 xi_half = grid["xi_half"]
271 khat = interface_conductivity(C_it, props) # 长 M
272 C_arr = np.asarray(C_it, dtype=float)
273 rho_cp = np.broadcast_to(props.rho(C_arr) * props.cp(C_arr), C_arr.shape) # 长 M+1
274 coef_face = 2.0 * one_over_R2 * xi_half * khat / dxi
275 a_sub = np.zeros(M + 1)
276 b_diag = np.zeros(M + 1)
277 c_sup = np.zeros(M + 1)
278 d_rhs = np.zeros(M + 1)
279 for j in range(M + 1):
280     rc = rho_cp[j]
281     b = w[j] * rc / dt
282     rhs = w[j] * rc / dt * T_old[j]
283     if j >= 1:
284         f_in = coef_face[j - 1]
285         a_sub[j] = -f_in
286         b += f_in
287     if j <= M - 1:
288         f_out = coef_face[j]
289         c_sup[j] = -f_out
290         b += f_out
291     else:
292         surf = 2.0 * h / R_now
293         b += surf
294         rhs += surf * Tair
295         rhs -= latent_sink # 表面蒸发吸热 (负源)
296     b_diag[j] = b
297     d_rhs[j] = rhs
298 return a_sub, b_diag, c_sup, d_rhs
299
300 # =====
301 # 主时间推进: theta=1 向后欧拉 + Picard 内迭代 (T 与 C 同一循环, M4/M5)
302 # =====
303 def solve_coupled(props, t_end, dt, env, *, mode="hold", R_interp=None,
304                  R_tmax=None, latent=0.0, R0=P.R0, C0=P.C0, T0=P.T0,
305                  M=P.M_GRID, endpoint_stop=False, record_full=False,
306                  interface="kirchhoff"):
307     """求解耦合热-质传递, 向后欧拉 + Picard 内迭代。
308
309     interface: "kirchhoff"(主算) / "arithmetic" / "harmonic" (仅灵敏度对照, sec.8.4)。
310     endpoint_stop=True: Cmax 首次 < C_TH 即停 (终点反演, 省算)。
311     record_full=True: 记录每步 T、C 全场 (导出用)。
312     返回 dict: t(输出时刻)、T_hist、C_hist(若 record_full)、t_star_s、t_star_h、
313     cmax_cross、cmax_prev、cbar0、cbar_end、cum_flux、cons_residual、

```

```

314     picard_res_max, n_steps, R_hist, Cs_end, Cair_end.
315     """
316     grid = build_grid(M)
317     w = grid["w_j"]
318     C = np.full(M + 1, C0)
319     T = np.full(M + 1, T0)
320     cbar0 = float(np.dot(w, C))
321     n_steps = int(round(t_end / dt))
322     t_out = [0.0]
323     T_hist = [T.copy()] if record_full else None
324     C_hist = [C.copy()] if record_full else None
325     R_hist = [R0]
326     cbar = cbar0
327     cum_flux = 0.0
328     cons_res = 0.0
329     picard_res_max = 0.0
330     cmax_prev = float(C.max())
331     t_star_s = None
332     cmax_cross = None
333     cs_end = float(C[-1])
334     cair_end = env_at(0.0, env, mode)[1]
335     for n in range(1, n_steps + 1):
336         tn1 = n * dt
337         Tair, Cair = env_at(tn1, env, mode)
338         R_now = R0 if R_interp is None else radius_at(R_interp, tn1, R_tmax)
339         one_over_R2 = 1.0 / (R_now * R_now)
340         C_old, T_old = C, T
341         C_it, T_it = C.copy(), T.copy()
342         res = np.inf
343         rho_s_R2 = None # M7 不变量占位 (收缩守恒基准)
344         for it in range(P.PICARD_MAX):
345             if interface == "kirchhoff":
346                 aC, bC, cC, dC = _assemble_moisture(
347                     C_old, C_it, T_it, props, grid, one_over_R2, R_now, Cair, P.HM_CONV, dt)
348             else:
349                 aC, bC, cC, dC = _assemble_moisture_alt(
350                     C_old, C_it, T_it, props, grid, one_over_R2, R_now, Cair,
351                     P.HM_CONV, dt, interface)
352             C_new = thomas_solve(aC, bC, cC, dC)
353             if latent != 0.0:
354                 rho_s = props.rho(C_it[-1]) / (1.0 + C_it[-1])
355                 latent_sink = 2.0 / R_now * latent * rho_s * P.HM_CONV * (C_it[-1] - Cair)
356             else:
357                 latent_sink = 0.0
358             aT, bT, cT, dT = _assemble_temperature(
359                 T_old, C_it, T_it, props, grid, one_over_R2, R_now, Tair, P.H_CONV, dt,
                 latent_sink)

```

```

360     T_new = thomas_solve(aT, bT, cT, dT)
361     res = max(np.max(np.abs(C_new - C_it)), np.max(np.abs(T_new - T_it)))
362     C_it, T_it = C_new, T_new
363     if res < P.PICARD_TOL:
364         break
365     picard_res_max = max(picard_res_max, res)
366     C, T = C_it, T_it
367     # 守恒审计 (构造性精确恒等, 逐步核对)
368     cbar_new = float(np.dot(w, C))
369     step_flux = dt * (2.0 * P.HM_CONV / R_now) * (C[-1] - Cair)
370     cum_flux += step_flux
371     imbalance = abs((cbar_new - cbar) + step_flux) / cbar0
372     cons_res = max(cons_res, imbalance)
373     cbar = cbar_new
374     # 物理约束
375     assert C.min() > -1e-6, f"RC7 违反: 出现负含水率 {C.min():.3e} at t={tn1}"
376     rho_s_R2 = (props.rho(C0) / (1.0 + C0)) * R_now * R_now # 记录不变量口径(M7)
377     cmax_new = float(C.max())
378     if record_full:
379         T_hist.append(T.copy())
380         C_hist.append(C.copy())
381         t_out.append(tn1)
382         R_hist.append(R_now)
383     # 首次穿越检测 (空间最大值判据, sec.7.6)
384     if t_star_s is None and cmax_prev >= P.C_TH and cmax_new < P.C_TH:
385         frac = (cmax_prev - P.C_TH) / (cmax_prev - cmax_new)
386         t_star_s = (n - 1) * dt + dt * frac
387         cmax_cross = cmax_new
388         cs_end = float(C[-1])
389         cair_end = Cair
390         if endpoint_stop:
391             break
392     cmax_prev = cmax_new
393     return {
394         "t": np.array(t_out), "T_hist": T_hist, "C_hist": C_hist,
395         "R_hist": np.array(R_hist), "t_star_s": t_star_s,
396         "t_star_h": (t_star_s / P.H_TO_S) if t_star_s is not None else None,
397         "cmax_cross": cmax_cross, "cmax_prev": cmax_prev,
398         "cbar0": cbar0, "cbar_end": cbar, "cum_flux": cum_flux,
399         "cons_residual": cons_res, "picard_res_max": picard_res_max,
400         "n_steps": n_steps, "Cs_end": cs_end, "Cair_end": cair_end,
401         "grid": grid, "argmax_r_end": int(np.argmax(C)),
402         "rho_s_R2": rho_s_R2,
403     }
404
405
406 def _assemble_moisture_alt(C_old, C_it, T_it, props, grid, one_over_R2, R_now,

```

```

407         Cair, hm, dt, interface):
408     """对照用界面平均（算术/调和），仅供 sec.8.4 离散决策可证伪性验证，非主算路径。
409
410     interface="arithmetic": 算术平均 0.5(D_L+D_R);
411     interface="harmonic": 调和平均 2 D_L D_R/(D_L+D_R)（报告禁用，此处仅量化其后果）。
412     """
413     M = grid["M"]
414     w = grid["w_j"]
415     dxi = grid["dxi"]
416     xi_half = grid["xi_half"]
417     a = props.d_exp
418     Cc = np.maximum(C_it, P.CFLOOR)
419     D_node = props.d_pre * arrhenius_factor(T_it, props.arr_T) * np.exp(-a / Cc)
420     DL, DR = D_node[:-1], D_node[1:]
421     if interface == "arithmetic":
422         Dhat = 0.5 * (DL + DR)
423     elif interface == "harmonic":
424         Dhat = 2.0 * DL * DR / (DL + DR)
425     else:
426         raise ValueError(interface)
427     coef_face = 2.0 * one_over_R2 * xi_half * Dhat / dxi
428     a_sub = np.zeros(M + 1)
429     b_diag = np.zeros(M + 1)
430     c_sup = np.zeros(M + 1)
431     d_rhs = np.zeros(M + 1)
432     for j in range(M + 1):
433         b = w[j] / dt
434         rhs = w[j] / dt * C_old[j]
435         if j >= 1:
436             a_sub[j] = -coef_face[j - 1]
437             b += coef_face[j - 1]
438         if j <= M - 1:
439             c_sup[j] = -coef_face[j]
440             b += coef_face[j]
441         else:
442             surf = 2.0 * hm / R_now
443             b += surf
444             rhs += surf * Cair
445         b_diag[j] = b
446         d_rhs[j] = rhs
447     return a_sub, b_diag, c_sup, d_rhs
448 # =====
449 # 输出场映射: xi 网格解 -> 固定物理列 0.0..2.0 cm (M8 线性插值, P4 域外留空)
450 # =====
451 def map_field_to_columns(field, R_now, *, shrinking=False):
452     """把一时刻 xi 网格解（长 M+1, 节点 xi=0..1）映射到 21 个固定物理列。
453

```

```

454 固定域 (P1--P3, R_now==R0 恒定): M=20 时节点与列严格重合, 返回原值 (零插值)。
455 收缩域 (P4, shrinking=True): 物理列 p_cm=0..1.9 映到 xi=(p_cm*CM_TO_M)/R_now,
456 线性插值; xi>1 (物理半径已收缩到该列内侧) 写 np.nan; 末列(索引20)恒取表面 xi=1 值。
457 返回长度 21 的一维数组。
458 """
459 field = np.asarray(field, dtype=float)
460 M = field.shape[0] - 1
461 xi_nodes = np.linspace(0.0, 1.0, M + 1)
462 ncol = P.N_COLS
463 out = np.full(ncol, np.nan)
464 if not shrinking:
465     # 物理列 j*DR_OUT 对应 xi=(j*DR_OUT_CM*CM_TO_M)/R0; M=20 时恰为节点
466     for j in range(ncol):
467         xi_t = (P.R_OUT_CM[j] * P.CM_TO_M) / R_now
468         out[j] = np.interp(xi_t, xi_nodes, field)
469     return out
470 for j in range(ncol - 1):
471     xi_t = (P.R_OUT_CM[j] * P.CM_TO_M) / R_now
472     out[j] = np.interp(xi_t, xi_nodes, field) if xi_t <= 1.0 else np.nan
473 out[ncol - 1] = field[-1] # 末列=当前物理表面 xi=1
474 return out
475 def build_output_matrix(hist, t_out, R_hist, *, shrinking=False,
476                        t_star_s=None, cross_step=None):
477     """由 record_full 的场历史构造输出矩阵 (n_rows x 21)。
478
479     hist: 各时刻场列表 (长 n_out+1, 含 t=0); t_out/R_hist 同步。
480     终点定位 (P3/P4, 给 t_star_s 与 cross_step=n): 输出规则步 0..(n-1)*dt 的整行,
481     再把末行时间替换为 t*, 场按穿越前后两步 [n-1,n] 线性插值 (frac 定位)。
482     返回 (time_col 长度 n_rows, matrix n_rowsx21)。
483     """
484     rows = []
485     times = []
486     if t_star_s is None:
487         for k in range(len(hist)):
488             rows.append(map_field_to_columns(hist[k], R_hist[k], shrinking=shrinking))
489             times.append(t_out[k])
490         return np.array(times), np.vstack(rows)
491     n = cross_step
492     for k in range(n): # 规则步 0..n-1 (含 t=0 初值行)
493         rows.append(map_field_to_columns(hist[k], R_hist[k], shrinking=shrinking))
494         times.append(t_out[k])
495     dt = t_out[1] - t_out[0]
496     frac = (t_star_s - (n - 1) * dt) / dt
497     field_star = (1.0 - frac) * np.asarray(hist[n - 1]) + frac * np.asarray(hist[n])
498     R_star = (1.0 - frac) * R_hist[n - 1] + frac * R_hist[n]
499     rows.append(map_field_to_columns(field_star, R_star, shrinking=shrinking))
500     times.append(t_star_s)

```

```

501     return np.array(times), np.vstack(rows)
502 # =====
503 # Excel 导出: 表头逐字对齐附件3 模板; 四位小数; sheet_name 显式
504 # =====
505 def _columns_header(last_surface=False):
506     """构造 22 列表头: A 列时间 + 21 个径向列 (0.0..2.0 cm)。
507     last_surface=True 时末列语义替换为「药材表面」(result4, R7)。"""
508     cols = [TIME_HEADER] + [f"{r:.1f}" for r in P.R_OUT_CM]
509     if last_surface:
510         cols[-1] = SURFACE_LABEL
511     return cols
512
513
514 def _frame(times, matrix, last_surface=False):
515     """时间列 + 场矩阵 -> DataFrame (四位小数), 列名对齐模板。np.nan 落盘为空单元。"""
516     data = {TIME_HEADER: np.round(np.asarray(times, dtype=float), P.DECIMALS)}
517     hdr = _columns_header(last_surface)
518     mat = np.round(np.asarray(matrix, dtype=float), P.DECIMALS)
519     for j, name in enumerate(hdr[1:]):
520         data[name] = mat[:, j]
521     return pd.DataFrame(data, columns=hdr)
522
523
524 def write_two_sheet(path, times, T_mat, C_mat):
525     """result1/result2: 两工作表「温度」「水分浓度」(R6), 行=时刻, 列=径向。"""
526     with pd.ExcelWriter(path, engine="openpyxl") as xw:
527         _frame(times, T_mat).to_excel(xw, sheet_name="温度", index=False)
528         _frame(times, C_mat).to_excel(xw, sheet_name="水分浓度", index=False)
529
530
531 def write_one_sheet(path, times, C_mat, *, last_surface=False):
532     """result3/result4: 单表 Sheet1 存水分浓度 (R6/R7); result4 末列=药材表面。"""
533     with pd.ExcelWriter(path, engine="openpyxl") as xw:
534         _frame(times, C_mat, last_surface=last_surface).to_excel(
535             xw, sheet_name="Sheet1", index=False)
536 # =====
537 # 解析退化验证 (sec.8.1): 常物性圆柱 Robin 扩散 Bessel 级数
538 # =====
539 def bessell_roots(Bi, n_roots=40):
540     """求  $\mu \cdot J_1(\mu) = Bi \cdot J_0(\mu)$  的前 n_roots 个正根 (Robin 特征值,  $j_0/j_1$  由 scipy)。
541
542     在 [eps, N*pi] 上按符号变化 brentq 定位 (每个  $J_0$  零点区间恰一根)。
543     """
544     g = lambda m: m * j1(m) - Bi * j0(m)
545     roots = []
546     step = np.pi / 200.0
547     prev_m = 1e-6

```

```

548     prev_v = g(prev_m)
549     m = prev_m + step
550     while len(roots) < n_roots and m < (n_roots + 2) * np.pi:
551         v = g(m)
552         if np.isfinite(prev_v) and np.isfinite(v) and prev_v * v < 0.0:
553             roots.append(brentq(g, prev_m, m, xtol=1e-12, rtol=1e-14))
554             prev_m, prev_v = m, v
555             m += step
556     return np.array(roots)
557
558
559 def bessel_series_solution(xi, Fo, Bi, n_roots=40):
560     """无量纲 theta(xi,Fo)=? 2Bi/((mu_n^2+Bi^2)J0(mu_n))*J0(mu_n xi)*e^{-mu_n^2 Fo}。
561
562     theta=(C-C_air)/(C0-C_air), xi=r/R, Fo=D t/R^2。返回 xi 处的解（可数组）。
563     """
564     mus = bessel_roots(Bi, n_roots)
565     xi = np.asarray(xi, dtype=float)
566     theta = np.zeros_like(xi, dtype=float)
567     for mu in mus:
568         coef = 2.0 * Bi / ((mu ** 2 + Bi ** 2) * j0(mu))
569         theta = theta + coef * j0(mu * xi) * np.exp(-(mu ** 2) * Fo)
570     return theta
571
572 def const_env(Tair, Cair, t_end):
573     """构造常值环境 (t,Tair,Cair)，供解析退化/复算测试用。"""
574     t = np.array([0.0, t_end])
575     return t, np.array([Tair, Tair]), np.array([Cair, Cair])
576
577
578 def analytic_verify(D_const=5e-9, t_end=3600.0, dt=1.0, C0=2.55, Cair=0.05,
579                     n_roots=60, M=P.M_GRID):
580     """常物性圆柱 Robin 扩散：FVM 数值解 vs Bessel 解析级数 (sec.8.1)。
581
582     D 恒定 (d_exp=0, arr_T=0, 用 interface='arithmetic' 绕开 expl(0))。
583     返回 dict: max_abs_err、rel_err_l2、Bi、Fo（内部节点，排除表面奇异层）。
584     """
585     props = Properties(P.rho_a2, P.cp_a2, P.k_a2, D_const, 0.0, 0.0)
586     env = const_env(P.T0, Cair, t_end)
587     r = solve_coupled(props, t_end, dt, env, mode="hold", R0=P.R0, C0=C0,
588                       T0=P.T0, M=M, record_full=True, interface="arithmetic")
589     C_num = r["C_hist"][-1]
590     xi = np.linspace(0.0, 1.0, M + 1)
591     Bi = P.HM_CONV * P.R0 / D_const
592     Fo = D_const * t_end / (P.R0 * P.R0)
593     theta = bessel_series_solution(xi, Fo, Bi, n_roots)
594     C_ana = Cair + (C0 - Cair) * theta
595     interior = slice(0, M) # 排除表面节点（级数在 xi=1 收敛慢）

```

```

595     err = np.abs(C_num[interior] - C_ana[interior])
596     denom = np.linalg.norm(C_ana[interior])
597     return {
598         "max_abs_err": float(err.max()),
599         "rel_err_l2": float(np.linalg.norm(err) / denom),
600         "Bi": float(Bi), "Fo": float(Fo),
601         "C_num_surface": float(C_num[-1]), "C_ana_surface": float(C_ana[-1]),
602     }
603 # =====
604 # 独立复算 (M9, CK10): solve_ivp/BDF 对半离散水分 ODE 交叉验证
605 # =====
606 def _moisture_rhs(C, props, grid, one_over_R2, R_now, Cair):
607     """半离散水分方程右端  $dC/dt = \text{flux}_j / w_j$  (与 _assemble_moisture 同一通量, 显式形式)。"""
608     M = grid["M"]
609     w = grid["w_j"]
610     dxi = grid["dxi"]
611     xi_half = grid["xi_half"]
612     T_dummy = np.full(M + 1, P.T0) # arr_T=0 时 D 与 T 无关
613     Dhat = interface_diffusivity(C, T_dummy, props)
614     coef_face = 2.0 * one_over_R2 * xi_half * Dhat / dxi # 长 M
615     flux = np.zeros(M + 1)
616     for j in range(M + 1):
617         if j >= 1:
618             flux[j] += coef_face[j - 1] * (C[j - 1] - C[j])
619         if j <= M - 1:
620             flux[j] += coef_face[j] * (C[j + 1] - C[j])
621         else:
622             flux[j] += (2.0 * P.HM_CONV / R_now) * (Cair - C[j])
623     return flux / w
624
625
626 def bdf_cross_check(props, t_end, env, C_theta_hist, t_hist, *, R0=P.R0,
627                    CO=P.CO, M=P.M_GRID, mode="hold"):
628     """对 P1 水分场用 BDF 独立复算, 返回与 theta=1 场的 max|dC| (CK10, 判据<1e-3)。
629
630     C_theta_hist/t_hist 为主求解器 record_full 输出 (含 t=0)。仅对 arr_T=0 解耦场适用。
631     """
632     grid = build_grid(M)
633     one_over_R2 = 1.0 / (R0 * R0)
634
635     def rhs(t, C):
636         Cair = env_at(float(t), env, mode)[1]
637         return _moisture_rhs(C, props, grid, one_over_R2, R0, Cair)
638
639     C_init = np.full(M + 1, CO)
640     sol = solve_ivp(rhs, (0.0, t_end), C_init, method="BDF",
641                    t_eval=np.asarray(t_hist, dtype=float),

```

```

642         rtol=1e-9, atol=1e-11, max_step=50.0)
643     C_bdf = sol.y.T # (n_out, M+1)
644     C_ref = np.vstack(C_theta_hist)
645     return float(np.max(np.abs(C_bdf - C_ref)))
646 # =====
647 # 空间收敛阶 (CK8) 与界面平均对照 (CK7)
648 # =====
649 def spatial_convergence(props, env, dt, *, grids=(20, 40, 80), R0=P.R0,
650                        C0=P.C0, T0=P.T0, t_cap=None):
651     """在三套网格上求 t*, Richardson 估计空间收敛阶 p=log2(|u20-u40|/|u40-u80|)。
652
653     返回 dict: t_star_h (各网格)、order_p。endpoint_stop 省算。
654     """
655     ts = {}
656     for M in grids:
657         r = solve_coupled(props, t_cap or 400.0 * P.H_T0_S, dt, env,
658                          mode="hold", R0=R0, C0=C0, T0=T0, M=M,
659                          endpoint_stop=True)
660         ts[M] = r["t_star_h"]
661     g = list(grids)
662     num = abs(ts[g[0]] - ts[g[1]])
663     den = abs(ts[g[1]] - ts[g[2]])
664     p = float(np.log2(num / den)) if den > 0 else float("nan")
665     return {"t_star_h": ts, "order_p": p}
666
667
668 def interface_comparison(props, env, dt, *, R0=P.R0, C0=P.C0, T0=P.T0,
669                        t_cap=None):
670     """同一模型仅改界面平均方式求 t*: Kirchhoff (主算) / 算术 / 调和 (CK7、sec.8.4)。
671
672     调和平均在 D 跨量级时被最小值锁死 -> t* 暴涨, 作可证伪对照 (报告禁用)。
673     """
674     out = {}
675     for name in ("kirchhoff", "arithmetic", "harmonic"):
676         r = solve_coupled(props, t_cap or 700.0 * P.H_T0_S, dt, env,
677                          mode="hold", R0=R0, C0=C0, T0=T0,
678                          endpoint_stop=True, interface=name)
679         out[name] = r["t_star_h"]
680     return out
681 # =====
682 # 每 6 h 中心含水率序列 (CK11/CK12) 与表面通量匹配审计
683 # =====
684 def center_every_6h(C_hist, t_out, t_star_s):
685     """从场历史抽 t=6,12,... h (不超过 t*) 的中心含水率 C(r=0)。返回 (times_h, C_center)。"""
686     C_arr = np.vstack(C_hist)
687     t = np.asarray(t_out, dtype=float)
688     marks_h = []

```

```

689     vals = []
690     k = 1
691     while k * 6.0 * P.H_TO_S <= (t_star_s if t_star_s else t[-1]) + 1e-9:
692         tq = k * 6.0 * P.H_TO_S
693         c0 = float(np.interp(tq, t, C_arr[:, 0]))
694         marks_h.append(k * 6.0)
695         vals.append(c0)
696         k += 1
697     return marks_h, vals
698
699
700 def surface_cv_residual(C_new, C_old, T_new, props, env, dt, tn1, *,
701                        R0=P.R0, mode="hold"):
702     """表面控制体离散平衡残差 (P1-C2 falsifiable (a)): 在收敛解上核对
703
704         
$$w_M(C_M^n - C_M^{n-1})/dt = (2/R^2)xi_{M-1/2}DdC/dxi - (2 h_m/R)(C_M - C_{air})$$

705
706     左端为累积项、右端为「内部扩散通量+表面对流通量」。相对残差应达机器精度--
707     退化为直角坐标(缺 r 权)或第一类边界(C_M=C_air 直接赋值)时该恒等必然被破坏。
708     返回相对残差 |LHS-RHS|/max(|各项|)。
709     """
710     grid = build_grid(P.M_GRID)
711     Cair = env_at(tn1, env, mode)[1]
712     one_over_R2 = 1.0 / (R0 * R0)
713     Dhat = interface_diffusivity(C_new, T_new, props)
714     xi_half = grid["xi_half"]
715     dxi = grid["dxi"]
716     w_M = grid["w_j"][-1]
717     accum = w_M * (C_new[-1] - C_old[-1]) / dt
718     F_diff = 2.0 * one_over_R2 * xi_half[-1] * Dhat[-1] * (C_new[-2] - C_new[-1]) / dxi
719     F_conv = (2.0 * P.HM_CONV / R0) * (C_new[-1] - Cair)
720     denom = max(abs(accum), abs(F_diff), abs(F_conv), 1e-30)
721     return abs(accum - (F_diff - F_conv)) / denom
722
723
724 def picard_residual_trace(props, env, dt, *, tn1=None, R0=P.R0, C0=P.C0,
725                          T0=P.T0, mode="hold", n_warm=1):
726     """单个时间步内 Picard 残差序列 (P2-C2 falsifiable (c): 应单调下降至 <1e-11)。
727
728     先推进 n_warm 步建立非平凡场, 再在下一步记录每次内迭代的 max|d| 序列。
729     返回 (residuals list, n_iter, k_field_dispersion): k 径向相对离散度 (>1e-6 证明逐点物性)。
730     """
731     grid = build_grid(P.M_GRID)
732     C = np.full(P.M_GRID + 1, C0)
733     T = np.full(P.M_GRID + 1, T0)
734     for n in range(1, n_warm + 1):
735         tq = n * dt

```

```

736     Tair, Cair = env_at(tq, env, mode)
737     one_over_R2 = 1.0 / (R0 * R0)
738     C_old, T_old = C, T
739     C_it, T_it = C.copy(), T.copy()
740     for _ in range(P.PICARD_MAX):
741         aC, bC, cC, dC = _assemble_moisture(C_old, C_it, T_it, props, grid,
742                                             one_over_R2, R0, Cair, P.HM_CONV, dt)
743         C_new = thomas_solve(aC, bC, cC, dC)
744         aT, bT, cT, dT = _assemble_temperature(T_old, C_it, T_it, props, grid,
745                                             one_over_R2, R0, Tair, P.H_CONV, dt, 0.0)
746         T_new = thomas_solve(aT, bT, cT, dT)
747         res = max(np.max(np.abs(C_new - C_it)), np.max(np.abs(T_new - T_it)))
748         C_it, T_it = C_new, T_new
749         if res < P.PICARD_TOL:
750             break
751         C, T = C_it, T_it
752     # 记录步: 从收敛场再走一步, 逐迭代存残差
753     tq = (n_warm + 1) * dt
754     Tair, Cair = env_at(tq, env, mode)
755     one_over_R2 = 1.0 / (R0 * R0)
756     C_old, T_old = C, T
757     C_it, T_it = C.copy(), T.copy()
758     residuals = []
759     for _ in range(P.PICARD_MAX):
760         aC, bC, cC, dC = _assemble_moisture(C_old, C_it, T_it, props, grid,
761                                             one_over_R2, R0, Cair, P.HM_CONV, dt)
762         C_new = thomas_solve(aC, bC, cC, dC)
763         aT, bT, cT, dT = _assemble_temperature(T_old, C_it, T_it, props, grid,
764                                             one_over_R2, R0, Tair, P.H_CONV, dt, 0.0)
765         T_new = thomas_solve(aT, bT, cT, dT)
766         res = max(np.max(np.abs(C_new - C_it)), np.max(np.abs(T_new - T_it)))
767         residuals.append(float(res))
768         C_it, T_it = C_new, T_new
769         if res < P.PICARD_TOL:
770             break
771     kv = np.broadcast_to(props.k(C_it), C_it.shape)
772     k_disp = float((kv.max() - kv.min()) / max(abs(kv.mean()), 1e-30))
773     return residuals, len(residuals), k_disp

```

1.3 问题一 problem1.py

```

1  # -*- coding: utf-8 -*-
2  """问题1 驱动: 预热平衡阶段 (附录2 常物性, D 无温度项)。
3
4  时程 1800 s、步长 1 s; 导出 result1.xlsx (温度/水分浓度两表, 1801x21) 。

```

```

5  锚点: CK1/CK2 (t=1800 s 五点场); 解析退化 (Bessel) 与 BDF 独立复算 (CK10)。
6  """
7  from __future__ import annotations
8  import json
9  from pathlib import Path
10 import numpy as np
11 import params as P
12 import utils
13
14 ROOT = P.FIG_DIR.parent
15 RESULT_XLSX = ROOT / "result1.xlsx"
16 JSON_OUT = P.FIG_DIR / "problem_1_results.json"
17
18
19 def run_problem1(write=True):
20     env = utils.load_environment()
21     r = utils.solve_coupled(utils.PROPS_P1, P.T_END_P1, P.DT_P1, env,
22                             mode="hold", record_full=True)
23     times, C_mat = utils.build_output_matrix(r["C_hist"], r["t"], r["R_hist"])
24     _, T_mat = utils.build_output_matrix(r["T_hist"], r["t"], r["R_hist"])
25
26     # 结构性验证: 表面控制体离散平衡残差、常物性对 T/C 偏导 (P1-C2/P1-C3)
27     flux_res = utils.surface_cv_residual(
28         r["C_hist"][-1], r["C_hist"][-2], r["T_hist"][-1], utils.PROPS_P1,
29         env, P.DT_P1, P.T_END_P1)
30     d_lo = utils.PROPS_P1.d_pre * np.exp(-utils.PROPS_P1.d_exp / P.CO) \
31         * utils.arrhenius_factor(P.TO - 10.0, utils.PROPS_P1.arr_T)
32     d_hi = utils.PROPS_P1.d_pre * np.exp(-utils.PROPS_P1.d_exp / P.CO) \
33         * utils.arrhenius_factor(P.TO + 10.0, utils.PROPS_P1.arr_T)
34     dD_dT_rel = abs(d_hi - d_lo) / d_lo # P1-C3: 应 ~0
35     D_CO = utils.PROPS_P1.d_pre * np.exp(-utils.PROPS_P1.d_exp / P.CO)
36
37     # 解析退化 + BDF 独立复算
38     av = utils.analytic_verify()
39     bdf_diff = utils.bdf_cross_check(utils.PROPS_P1, P.T_END_P1, env,
40                                     r["C_hist"], r["t"])
41
42     if write:
43         utils.write_two_sheet(RESULT_XLSX, times, T_mat, C_mat)
44     out = {
45         "problem": 1,
46         "p1_T_1800_5pt": [round(float(T_mat[-1, i]), 4) for i in (0, 5, 10, 15, 20)],
47         "p1_C_1800_5pt": [round(float(C_mat[-1, i]), 4) for i in (0, 5, 10, 15, 20)],
48         "p1_conservation_residual": r["cons_residual"],
49         "p1_picard_residual_max": r["picard_res_max"],
50         "p1_surface_cv_residual": float(flux_res),
51         "p1_dD_dT_rel_perturb": float(dD_dT_rel),

```

```

52     "p1_D_at_C0": float(D_C0),
53     "p1_bessel_rel_err_l2": av["rel_err_l2"],
54     "p1_bessel_max_abs_err": av["max_abs_err"],
55     "p1_bdf_cross_check_max_abs_diff": float(bdf_diff),
56     "p1_n_rows": int(times.shape[0]),
57     "p1_discrete_measure_sum_error": float(abs(r["grid"]["w_j"].sum() - 1.0)),
58 }
59 if write:
60     P.FIG_DIR.mkdir(parents=True, exist_ok=True)
61     JSON_OUT.write_text(json.dumps(out, ensure_ascii=False, indent=2),
62                          encoding="utf-8")
63 out["_solve"] = r
64 return out
65
66
67 if __name__ == "__main__":
68     o = run_problem1()
69     print("P1 done:", {k: v for k, v in o.items() if k != "_solve"})

```

1.4 问题二 problem2.py

```

1  # -*- coding: utf-8 -*-
2  """问题2 驱动：预热+恒温全过程（附录3 变物性，D(C,T) Arrhenius，开尔文）。
3
4  时程 3 h、步长 1 s；导出 result2.xlsx（温度/水分浓度两表，10801x21）。
5  锚点：CK3/CK4（t=3 h 五点场）。P2-C2 证据：k 径向离散、Picard 单调下降、开尔文入口。
6  """
7  from __future__ import annotations
8  import json
9  import numpy as np
10 import params as P
11 import utils
12
13 ROOT = P.FIG_DIR.parent
14 RESULT_XLSX = ROOT / "result2.xlsx"
15 JSON_OUT = P.FIG_DIR / "problem_2_results.json"
16
17
18 def run_problem2(write=True):
19     env = utils.load_environment()
20     r = utils.solve_coupled(utils.PROPS_P23, P.T_END_P2, P.DT_P2, env,
21                             mode="hold", record_full=True)
22     times, C_mat = utils.build_output_matrix(r["C_hist"], r["t"], r["R_hist"])
23     _, T_mat = utils.build_output_matrix(r["T_hist"], r["t"], r["R_hist"])
24

```

```

25 # P2-C2 falsifiable: Picard 残差单调下降、k 逐点随 C 变、开尔文入口
26 resids, n_iter, k_disp = utils.picard_residual_trace(
27     utils.PROPS_P23, env, P.DT_P2, n_warm=60)
28 monotone = all(resids[i] >= resids[i + 1] for i in range(len(resids) - 1))
29 # 终点场 k 径向离散度 (同一时刻不同 r 处 k 不等 -> 排除冻结常数)
30 kv = np.broadcast_to(utils.PROPS_P23.k(r["C_hist"][-1]), (P.M_GRID + 1,))
31 k_disp_end = float((kv.max() - kv.min()) / abs(kv.mean()))
32 surf_res = utils.surface_cv_residual(
33     r["C_hist"][-1], r["C_hist"][-2], r["T_hist"][-1], utils.PROPS_P23,
34     env, P.DT_P2, P.T_END_P2)
35
36 if write:
37     utils.write_two_sheet(RESULT_XLSX, times, T_mat, C_mat)
38 out = {
39     "problem": 2,
40     "p2_T_3h_5pt": [round(float(T_mat[-1, i]), 4) for i in (0, 5, 10, 15, 20)],
41     "p2_C_3h_5pt": [round(float(C_mat[-1, i]), 4) for i in (0, 5, 10, 15, 20)],
42     "p2_conservation_residual": r["cons_residual"],
43     "p2_picard_residual_max": r["picard_res_max"],
44     "p2_picard_trace": resids,
45     "p2_picard_n_iter": n_iter,
46     "p2_picard_monotone": bool(monotone),
47     "p2_k_radial_dispersion": k_disp_end,
48     "p2_surface_cv_residual": float(surf_res),
49     "p2_n_rows": int(times.shape[0]),
50     "p2_discrete_measure_sum_error": float(abs(r["grid"]["w_j"].sum() - 1.0)),
51 }
52 if write:
53     P.FIG_DIR.mkdir(parents=True, exist_ok=True)
54     JSON_OUT.write_text(json.dumps(out, ensure_ascii=False, indent=2),
55                          encoding="utf-8")
56 out["_solve"] = r
57 return out
58
59
60 if __name__ == "__main__":
61     o = run_problem2()
62     print("P2 done:", {k: v for k, v in o.items()
63                       if k not in ("_solve", "p2_picard_trace")})

```

1.5 问题三 problem3.py

```

1 # -*- coding: utf-8 -*-
2 """问题3 驱动: 恒温烘干至中心达标 (附录3 变物性, 固定域)。
3

```

```

4  终点反演：空间最大含水率首次 <0.15 kg/kg (sec.7.6 首穿判据)，t*≈57.26 h。
5  步长 60 s；导出 result3.xlsx (单表水分浓度，末行=t* 线性插值场)。
6  P3-C2 证据：首穿在原始解上、argmax 近中心、C_max 单调降、C_surf>=C_air；
7  CK7 界面平均对照 (算术/调和)，CK8 空间收敛阶 (Richardson)，CK11 每 6 h 中心值。
8  """
9  from __future__ import annotations
10 import json
11 import numpy as np
12 import params as P
13 import utils
14
15 ROOT = P.FIG_DIR.parent
16 RESULT_XLSX = ROOT / "result3.xlsx"
17 JSON_OUT = P.FIG_DIR / "problem_3_results.json"
18 T_CAP = 400.0 * P.H_TO_S
19
20
21 def run_problem3(write=True):
22     env = utils.load_environment()
23     r = utils.solve_coupled(utils.PROPS_P23, T_CAP, P.DT_P34, env,
24                             mode="hold", endpoint_stop=True, record_full=True)
25     cross_step = len(r["C_hist"]) - 1
26     times, C_mat = utils.build_output_matrix(
27         r["C_hist"], r["t"], r["R_hist"],
28         t_star_s=r["t_star_s"], cross_step=cross_step)
29
30     # P3-C2 falsifiable: C_max 逐步单调降 (首穿唯一)、argmax 近中心、表面>=环境
31     cmax_series = [float(np.max(f)) for f in r["C_hist"]]
32     cmax_monotone = all(cmax_series[i] >= cmax_series[i + 1]
33                         for i in range(len(cmax_series) - 1))
34     c_surf_minus_air = float(r["Cs_end"] - r["Cair_end"])
35
36     # CK11 每 6 h 中心含水率；CK8 空间收敛阶；CK7 界面平均对照
37     marks_h, c_center = utils.center_every_6h(r["C_hist"], r["t"], r["t_star_s"])
38     sconv = utils.spatial_convergence(utils.PROPS_P23, env, P.DT_P34,
39                                       t_cap=T_CAP)
40     iface = utils.interface_comparison(utils.PROPS_P23, env, P.DT_P34,
41                                       t_cap=700.0 * P.H_TO_S)
42
43     if write:
44         utils.write_one_sheet(RESULT_XLSX, times, C_mat)
45     out = {
46         "problem": 3,
47         "p3_t_star_h": round(float(r["t_star_h"]), 6),
48         "p3_cmax_cross": float(r["cmax_cross"]),
49         "p3_cmax_prev": float(r["cmax_prev"]),
50         "p3_argmax_r_end": int(r["argmax_r_end"]),

```

```

51     "p3_cmax_monotone": bool(cmax_monotone),
52     "p3_C_surface_minus_C_air": c_surf_minus_air,
53     "p3-Cs_end": float(r["Cs_end"]),
54     "p3_Cair_end": float(r["Cair_end"]),
55     "p3_conservation_residual": r["cons_residual"],
56     "p3_picard_residual_max": r["picard_res_max"],
57     "p3_center_6h_marks_h": marks_h,
58     "p3_center_6h_values": [round(v, 6) for v in c_center],
59     "p3_spatial_order_p": round(float(sconv["order_p"]), 4),
60     "p3_spatial_t_star_h": {str(k): round(float(v), 6)
61                             for k, v in sconv["t_star_h"].items()},
62     "p3_interface_t_star_h": {k: round(float(v), 4)
63                               for k, v in iface.items()},
64     "p3_n_rows": int(times.shape[0]),
65     "p3_discrete_measure_sum_error": float(abs(r["grid"]["w_j"].sum() - 1.0)),
66 }
67 if write:
68     P.FIG_DIR.mkdir(parents=True, exist_ok=True)
69     JSON_OUT.write_text(json.dumps(out, ensure_ascii=False, indent=2),
70                          encoding="utf-8")
71 out["_solve"] = r
72 return out
73
74
75 if __name__ == "__main__":
76     o = run_problem3()
77     print("P3 done:", {k: v for k, v in o.items()
78                       if k not in ("_solve", "p3_center_6h_values",
79                                   "p3_center_6h_marks_h")})

```

1.6 问题四 problem4.py

```

1  # -*- coding: utf-8 -*-
2  """问题4 驱动：收缩域烘干至中心达标（附录4 变物性，动边界 Landau 定域）。
3
4  R(t) 由附件2 PCHIP 保形插值给定；xi=r/R(t) 定域，不变量 rho_s R^2=sigma,
5  漂移相消后余 1/R(t)^2 因子 (M7)。终点反演 t*~50.90 h。
6  步长 60 s；导出 result4.xlsx (单表，末列=药材表面；xi>1 处空单元)。
7  P4-C2 证据：R(t) 复现附件2<1e-8、单调非增、R(t*)=1.2 cm(CK6)、
8  CK12 每 6 h 中心值、加密步长 t* 等价(<0.2%)。
9  """
10 from __future__ import annotations
11 import json
12 import numpy as np
13 import params as P

```

```

14 import utils
15
16 ROOT = P.FIG_DIR.parent
17 RESULT_XLSX = ROOT / "result4.xlsx"
18 JSON_OUT = P.FIG_DIR / "problem_4_results.json"
19 T_CAP = 80.0 * P.H_TO_S
20
21
22 def run_problem4(write=True):
23     env = utils.load_environment()
24     pchip, t_arr, R_arr = utils.build_radius_interp()
25     R_tmax = float(t_arr[-1])
26
27     r = utils.solve_coupled(utils.PROPS_P4, T_CAP, P.DT_P34, env, mode="hold",
28                             R_interp=pchip, R_tmax=R_tmax,
29                             endpoint_stop=True, record_full=True)
30     cross_step = len(r["C_hist"]) - 1
31     times, C_mat = utils.build_output_matrix(
32         r["C_hist"], r["t"], r["R_hist"], shrinking=True,
33         t_star_s=r["t_star_s"], cross_step=cross_step)
34
35     # P4-C2 falsifiable: PCHIP 复现附件2 节点、单调非增、R(t*) (CK6)
36     R_pred = pchip(t_arr)
37     r_interp_err = float(np.max(np.abs(R_pred - R_arr) / np.abs(R_arr)))
38     r_max_pos_diff = float(np.max(np.diff(R_arr))) # 应 <=0 (非增)
39     R_tstar_cm = utils.radius_at(pchip, r["t_star_s"], R_tmax) / P.CM_TO_M
40
41     # CK12 每 6 h 中心含水率; 加密步长 t* 等价性 (PCHIP vs 更密输出)
42     marks_h, c_center = utils.center_every_6h(r["C_hist"], r["t"], r["t_star_s"])
43     r_dense = utils.solve_coupled(utils.PROPS_P4, T_CAP, P.DT_P34 / 2.0, env,
44                                     mode="hold", R_interp=pchip, R_tmax=R_tmax,
45                                     endpoint_stop=True)
46     t_star_dense_h = float(r_dense["t_star_h"])
47
48     if write:
49         utils.write_one_sheet(RESULT_XLSX, times, C_mat, last_surface=True)
50     out = {
51         "problem": 4,
52         "p4_t_star_h": round(float(r["t_star_h"]), 6),
53         "p4_t_star_dense_h": round(t_star_dense_h, 6),
54         "p4_cmax_cross": float(r["cmax_cross"]),
55         "p4_cmax_prev": float(r["cmax_prev"]),
56         "p4_argmax_r_end": int(r["argmax_r_end"]),
57         "p4_R_tstar_cm": round(float(R_tstar_cm), 4),
58         "p4_R_interp_rel_error_max": r_interp_err,
59         "p4_R_max_positive_diff": r_max_pos_diff,
60         "p4_rho_s_R2_invariant": float(r["rho_s_R2"]),

```

```

61     "p4_conservation_residual": r["cons_residual"],
62     "p4_picard_residual_max": r["picard_res_max"],
63     "p4_center_6h_marks_h": marks_h,
64     "p4_center_6h_values": [round(v, 6) for v in c_center],
65     "p4_n_rows": int(times.shape[0]),
66     "p4_discrete_measure_sum_error": float(abs(r["grid"]["w_j"].sum() - 1.0)),
67 }
68 if write:
69     P.FIG_DIR.mkdir(parents=True, exist_ok=True)
70     JSON_OUT.write_text(json.dumps(out, ensure_ascii=False, indent=2),
71                          encoding="utf-8")
72 out["_solve"] = r
73 return out
74
75
76 if __name__ == "__main__":
77     o = run_problem4()
78     print("P4 done:", {k: v for k, v in o.items()
79                       if k not in ("_solve", "p4_center_6h_values",
80                                   "p4_center_6h_marks_h")})

```